Министерство цифрового развития, связи и массовых коммуникаций РФ
Уральский технический институт связи и информатики (филиал)
ФГБОУ ВО «Сибирский государственный университет
телекоммуникаций и информатики» в г. Екатеринбурге
(УрТИСИ СибГУТИ)

Отчёт
по учебной практике
по МДК 02.02 «Инструментальные средства разработки
программного обеспечения»
на тему «Разработка приложения по варианту ДЭ
и реализация дипломного проекта»

студента IV курса 285 группы

Фамилия Каримов

Имя, отчество Вадим Эдуардович

Факультет Инфокоммуникаций, информатики и управления

Специальность 09.02.07 «Информационные системы
и программирование»

Екатеринбург, 2026

Отзыв руководителя

Содержание

Введение 5
1 Демонстрационный экзамен 6
1.1 Анализ предметной области и технического задания 6
1.2 Выбор стека технологий и инструментов разработки 7
1.3 Работа с базой данных 8
1.4 Практическая реализация решения 9
1.5 Интерфейс программы 12
1.6 Тестирование программы 14
1.7 Руководство системному программисту 15
1.7.1 Общие сведения о программе 15
1.7.2 Функции программы 15
1.7.3 Минимальный состав технических средств 16
1.7.4 Минимальный состав программных средств 16
1.7.5 Требования к персоналу (системному программисту) 16
1.8 Структура программы 17
1.8.2 Сведения о составных частях программы 17
1.8.2 Сведения о связях между составными частями программы 18
1.8.3 Сведения о связях с другими программами 18
1.9 Настройка программы 19
1.9.1 Настройка на состав технических средств 19
1.9.2 Настройка на состав программных средств 19
1.10 Проверка программы 20
1.10.1 Описание способов проверки 20
1.11 Сообщения системному программисту 20
2 Дипломный проект 22
2.1 Введение дипломного проекта 22
2.2 Анализ предметной области 23
2.3 Требование к своему программному продукту 24
2.4 Обоснование выбора инструментальных средств разработки 27
2.5 Разработка архитектуры блок-схем программы 27
2.6 Разработка макетов основных форм приложения 28
2.7 Проектирование и разработка базы данных 30
2.8 Разработка пользовательского интерфейса 33
2.9 Доработка приложения 48
2.9.1 Разработка блок-схем функций 48

2.9.2 Разработка функционала 51
Заключение 53
Библиография 54

Введение

В современном мире климатическое оборудование стало неотъемлемой частью комфортной жизнедеятельности человека. Кондиционеры, системы вентиляции, увлажнители и отопительные приборы требуют регулярного технического обслуживания и оперативного ремонта. Для компаний, предоставляющих такие услуги, важнейшей задачей является эффективная организация процесса приёма и обработки заявок, контроля сроков выполнения работ и качества обслуживания клиентов. Ручной учёт с использованием бумажных журналов или разрозненных электронных таблиц приводит к потере информации, задержкам в обработке и снижению уровня сервиса.

Автоматизация процесса учёта заявок позволяет не только упорядочить информацию, но и повысить прозрачность работы сервисной службы, сократить время реакции на обращения, оптимизировать распределение ресурсов и вести статистику для анализа эффективности деятельности.

Целью данной работы является разработка программного модуля для учёта заявок на ремонт климатического оборудования, обеспечивающего автоматизацию всех этапов жизненного цикла заявки: от регистрации до закрытия и формирования отчётности.

Для достижения данной цели необходимо выполнить следующие задачи:

- проанализировать предметную область и техническое задание, выделить основные сущности, входные и выходные данные, а также разработать алгоритмы обработки заявок (включая расчёт статистики) и представить их в виде блок-схем в соответствии с ГОСТ;
- спроектировать и создать базу данных в СУБД, приведя её к третьей нормальной форме, обеспечив ссылочную целостность и реализовав необходимые ограничения. Заполнить базу данных тестовой информацией и создать запросы для получения отчётности.;
- разработать веб-приложение на фреймворке Django, реализующее функционал учёта заявок: добавление, редактирование, просмотр, назначение ответственных, комментирование, заказ комплектующих, поиск и фильтрацию, а также расчёт статистики с визуализацией.;
- модифицировать систему, внедрив функционал для новой роли «Менеджер по качеству», чтобы оперативно решать проблемы с задержками ремонта;
- оформить проектную документацию, включая руководство системного программиста, в соответствии с установленными стандартами.

1 Демонстрационный экзамен

1.1 Анализ предметной области и технического задания

Предметная область «Учёт заявок на ремонт климатического оборудования» охватывает процессы взаимодействия сервисной службы с клиентами, нуждающимися в обслуживании и ремонте таких устройств, как кондиционеры, системы вентиляции, увлажнители и отопительное оборудование. Основная цель автоматизации – повышение эффективности работы сервисной организации, сокращение времени реакции на обращения, улучшение качества обслуживания и оптимизация использования ресурсов (специалистов, комплектующих).

Анализ описания предметной области позволил выделить ключевые бизнес-процессы:

- создание заявки: клиент оставляет информацию о неисправности (тип оборудования, модель, описание проблемы, контактные данные). Заявка может быть отредактирована клиентом на начальном этапе.;
- регистрация заявки: оператор сервисной службы принимает заявку, присваивает ей уникальный номер и фиксирует в системе. На этом этапе заявка получает статус «открыта»;
- обработка заявки: анализ поступившей информации, определение приоритета, назначение ответственного специалиста. При необходимости уточняются детали у заказчика;
- исполнение заявки: специалист выполняет ремонт, может заказывать комплектующие, оставлять комментарии. Статус заявки изменяется («в процессе ремонта», «ожидание комплектующих», «завершена»);
- отчётность и информирование: фиксация результатов ремонта, затраченных ресурсов, формирование отчётов для руководства и клиента;
- мониторинг и анализ – сбор статистики по времени выполнения, типам неисправностей, качеству работ для последующей оптимизации процессов.

В техническом задании определены функциональные и нефункциональные требования, а также требования к реализации. Функциональные требования включают:

- возможность добавления, редактирования и отслеживания заявок с набором обязательных полей (номер, дата, тип оборудования, модель, описание, ФИО заказчика, телефон, статус);
- изменение этапа выполнения, описания проблемы, ответственного специалиста;
- поиск и фильтрация заявок по номеру и параметрам;
- назначение ответственных, добавление комментариев и фиксация заказа комплектующих;
- расчёт статистики: количество выполненных заявок, среднее время ремонта, распределение по типам неисправностей.

1.2 Выбор стека технологий и инструментов разработки

В ходе разработки веб-приложения для учёта заявок на ремонт климатического оборудования был использован широкий спектр современных инструментов, обеспечивающих эффективную реализацию всех модулей проекта.

В качестве основного языка программирования выбран Python версии 3.13, который отличается простотой, богатой экосистемой и отличной поддержкой веб-разработки. На его основе развёрнут фреймворк Django 5.2, предоставляющий готовую архитектуру MTV (Model-Template-View), встроенную административную панель, механизм миграций и ORM для работы с базами данных. Django позволил быстро создать структуру проекта, организовать маршрутизацию, обработку форм и аутентификацию пользователей.

Для хранения данных использована реляционная СУБД PostgreSQL 15, которая обеспечивает надёжность, поддержку сложных запросов и ссылочной целостности. Взаимодействие с базой данных осуществлялось через Django ORM, а для прямого администрирования и выполнения резервного копирования применялись утилиты psql и pg_dump, входящие в состав PostgreSQL.

На уровне представлений и шаблонов активно применялись библиотеки для улучшения пользовательского интерфейса: Bootstrap 5 (с темой Bootswatch Flatly) обеспечил адаптивную вёрстку и стилизацию компонентов, а иконки Font Awesome добавили наглядности элементам управления. Для более сложных интерфейсных решений, таких как вкладки на детальной странице заявки и графики статистики, были задействованы стандартные возможности Bootstrap и библиотека Chart.js, позволившая визуализировать распределение заявок по типам оборудования. Кроме того, для улучшения выпадающих списков использовался плагин Select2.

Работа с формами была упрощена благодаря пакетам django-crispy-forms и crispy-bootstrap5, которые автоматически генерируют разметку, совместимую с Bootstrap, и выводят сообщения об ошибках. Для генерации QR-кодов, требуемых в дополнительном задании, использовалась библиотека qrcode в сочетании с Pillow для обработки изображений.

На этапе проектирования базы данных была создана ER-диаграмма, что позволило наглядно представить структуру сущностей и их отношения. Также применялись стандартные средства Django для генерации схемы базы данных через миграции.

Для импорта начальных данных (пользователей, комментариев, заявок) были разработаны кастомные management-команды Django, что обеспечило повторяемость и автоматизацию заполнения базы. Резервное копирование выполнялось с помощью pg_dump, создающего SQL-дампы для возможного восстановления.

1.3 Работа с базой данных

Для надежного хранения информации была спроектирована реляционная база данных под управлением СУБД PostgreSQL. В соответствии с заданием, главным требованием к разработке ER-модели стало приведение структуры к третьей нормальной форме (3НФ) и обеспечение строгой ссылочной целостности.

На основе анализа технического задания были выделены следующие сущности и их атрибуты:

- roles (роли): справочник системных ролей (roleID – первичный ключ (PK); roleName).
- statuses (статусы): справочник этапов выполнения работ (statusID – PK; statusName).
- users (пользователи): единая таблица для хранения данных всех участников системы, включая клиентов, мастеров и менеджеров (userID – PK; fio; phone; login; password; roleID – внешний ключ (FK)).
- requests (заявки): центральная таблица учета обращений (requestID – PK; startDate; climateTechType; climateTechModel; problemDescription; statusID – FK; completionDate; repairParts; masterID – FK; clientID – FK).
- comments (комментарии): таблица для фиксации дополнительных сведений и информации о заказанных комплектующих (commentID – PK; message; masterID – FK; requestID – FK).

Для обеспечения целостности данных связи между сущностями организованы по типу «один ко многим» (1:N) с использованием внешних ключей:

- роли и пользователи: одна роль может быть назначена нескольким пользователям;
- статусы и заявки: один статус может быть присвоен множеству заявок;
- пользователи (мастера) и заявки: один мастер может выполнять множество заявок;
- пользователи (клиенты) и заявки: один клиент может оставить множество заявок на ремонт;
- заявки и комментарии: одна заявка может иметь несколько комментариев от специалиста.

Разработанная логическая структура исключает дублирование данных, предотвращает аномалии обновления и обеспечивает высокую скорость выполнения запросов при формировании статистической отчетности.

ERD-диаграмма системы представлена на рисунке 1.1.

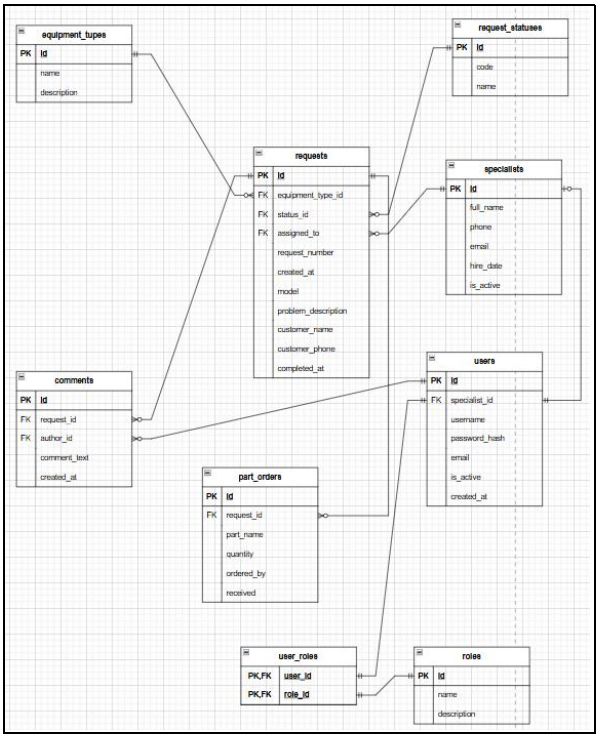


Рисунок 1.1 – ERD-диаграмма

Запрос создание базы данных представлен в приложении А.

1.4 Практическая реализация решения

В соответствии с требованиями технического задания, перед непосредственным написанием программного кода был проведен этап проектирования логики работы приложения. Для наглядной визуализации процессов обработки данных были разработаны графические алгоритмы (блок-схемы), выполненные в строгом соответствии с правилами оформления по ГОСТ 19.701.

В рамках проекта были спроектированы две основные блок-схемы:

Основной алгоритм учета заявок: Первая блок-схема описывает базовый алгоритм работы системы учета заявок на ремонт климатического оборудования. Она наглядно демонстрирует общий жизненный цикл обращения: получение входных данных от клиента при регистрации, проверку условий, изменение статусов заявки и формирование выходных данных при завершении ремонта (Рисунок 1.2)

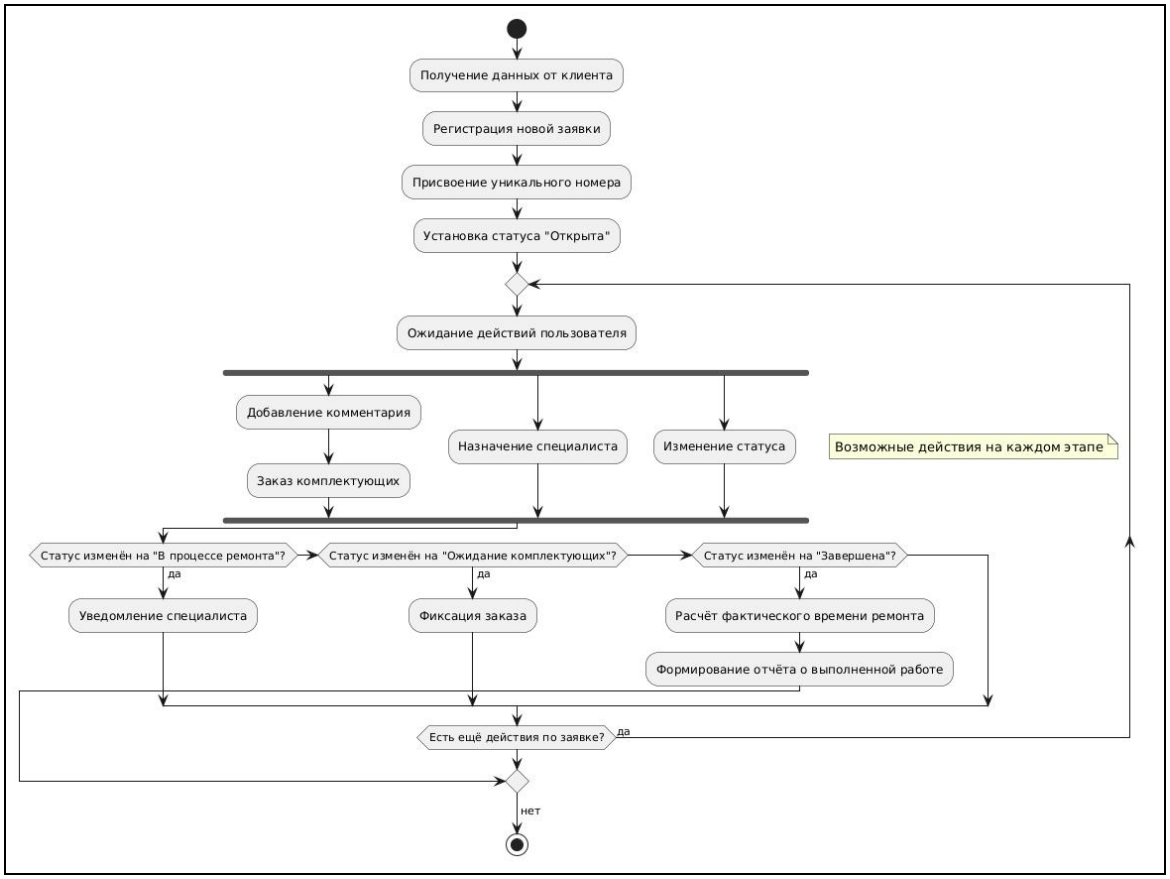


Рисунок 1.2 – Основная блок-схема

Детализированный алгоритм функции расчета: Вторая блок-схема выполняет глубокую детализацию одной из ключевых вычислительных функций модуля – расчета статистики работы отдела обслуживания. Данный алгоритм описывает последовательность циклов и условий, необходимых для подсчета количества выполненных заявок, вычисления среднего времени ремонта и группировки данных для статистики по типам неисправностей (Рисунок 1.3)

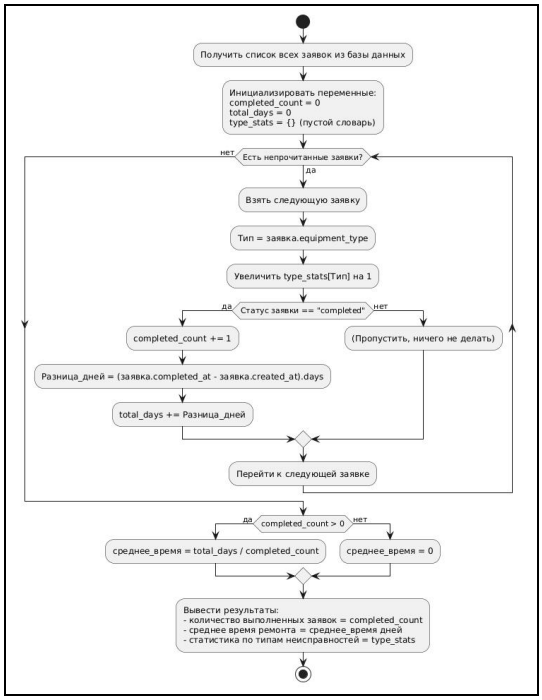


Рисунок 1.3 – Детализированная блок-схема

Программное обеспечение разработано в виде веб-приложения с использованием клиент-серверной архитектуры. Серверная часть реализована на языке Python с применением фреймворка Django версии 5.2, который обеспечивает удобную организацию кода по шаблону MTV (Model-Template-View) и включает встроенные механизмы для работы с базами данных, аутентификации, обработки форм и маршрутизации.

Для хранения данных используется реляционная СУБД PostgreSQL 15, с которой Django взаимодействует через встроенный ORM. Модели данных полностью соответствуют спроектированной ER-диаграмме и включают сущности: типы оборудования, статусы заявок, специалисты, пользователи, роли, заявки, комментарии и заказы комплектующих. Все связи реализованы с помощью внешних ключей, обеспечена ссылочная целостность и индексация часто используемых полей.

Основные функциональные блоки приложения:

- Аутентификация и регистрация пользователей с разделением на группы (оператор, специалист, менеджер по качеству). Использована встроенная модель User и группы Django.

- Управление заявками: создание, просмотр, редактирование, удаление. Формы созданы с помощью django-crispy-forms для единообразного отображения.

- Детальный просмотр заявки с возможностью добавления комментариев и заказа комплектующих.

- Поиск и фильтрация заявок по статусу, типу оборудования и текстовому запросу.

- Расчёт статистики: количество выполненных заявок, среднее время ремонта, распределение по типам оборудования. Данные выводятся в виде карточек и круговой диаграммы.

- Дополнительный функционал для менеджера: продление срока заявки (добавление комментария) и генерация QR-кода со ссылкой на форму обратной связи.

Для импорта начальных данных (пользователей, заявок, комментариев) созданы кастомные management-команды, которые автоматически заполняют базу при первом запуске. Это обеспечивает воспроизводимость среды и упрощает тестирование.

Безопасность приложения обеспечена на нескольких уровнях: все представления защищены декораторами @login_required, доступ к функциям ограничен через группы и кастомные разрешения (например, can_extend_deadline). Пароли пользователей хранятся в хешированном виде.

1.5 Интерфейс программы

Пользовательский интерфейс построен на базе шаблонов Django с использованием Bootstrap 5 (тема Bootswatch Flatly) для адаптивной и стильной вёрстки. Иконки Font Awesome улучшают визуальное восприятие. Для динамического отображения данных и интерактивности применены JavaScript-библиотеки: Chart.js для построения графиков статистики, Select2 для удобного выбора из выпадающих списков. Генерация QR-кодов выполнена с помощью библиотеки qrcode, изображения кодируются в base64 и встраиваются в HTML.

Интерфейс программы представлен на рисунках 1.4-1.9.

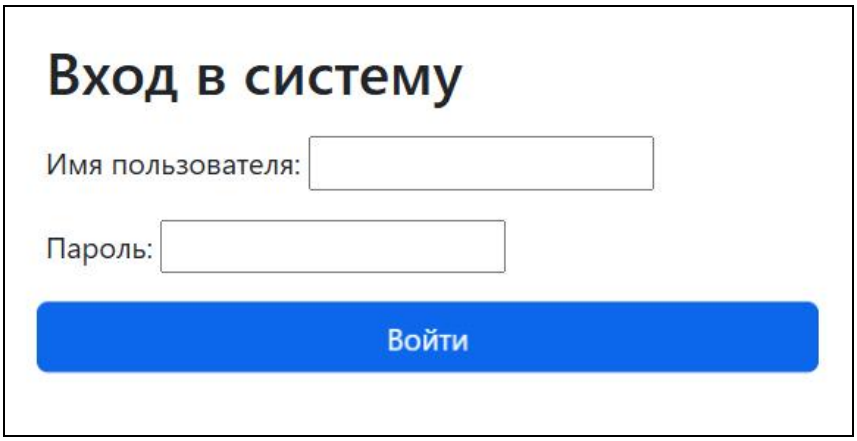


Рисунок 1.4 – Авторизация

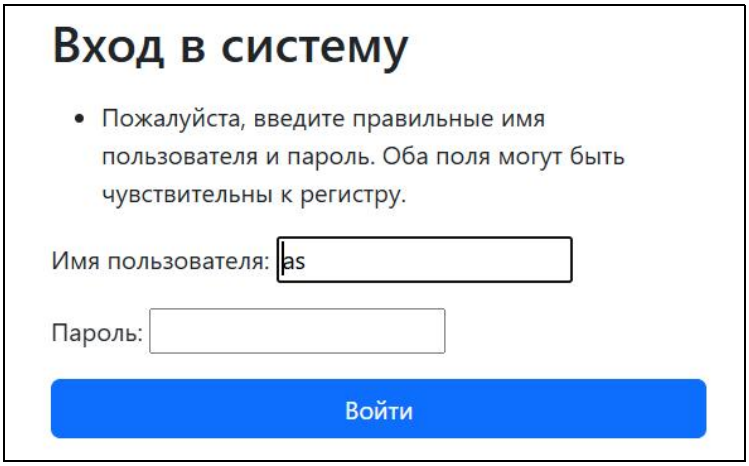


Рисунок 1.5 – Ошибка при введенных неверных данных

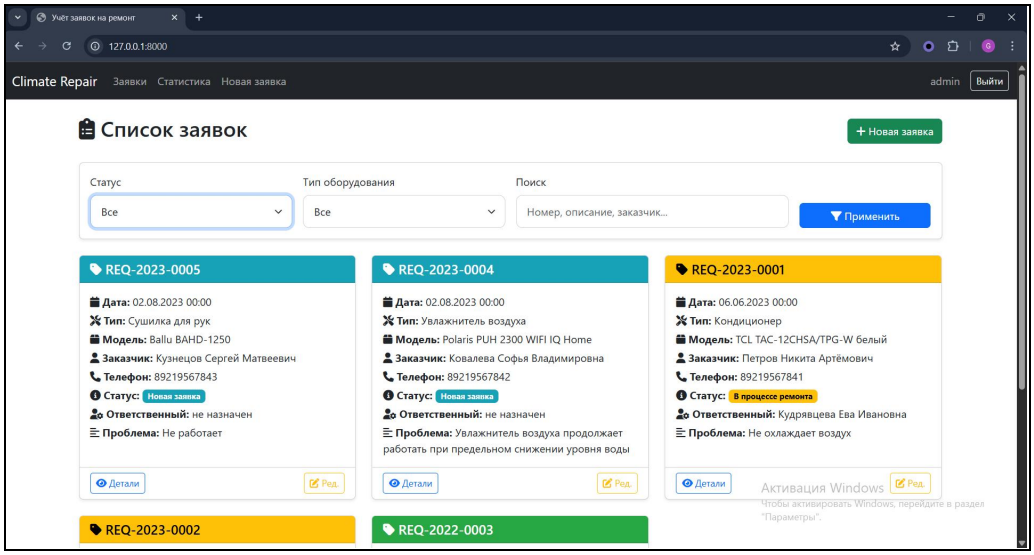


Рисунок 1.6 – Главная страница

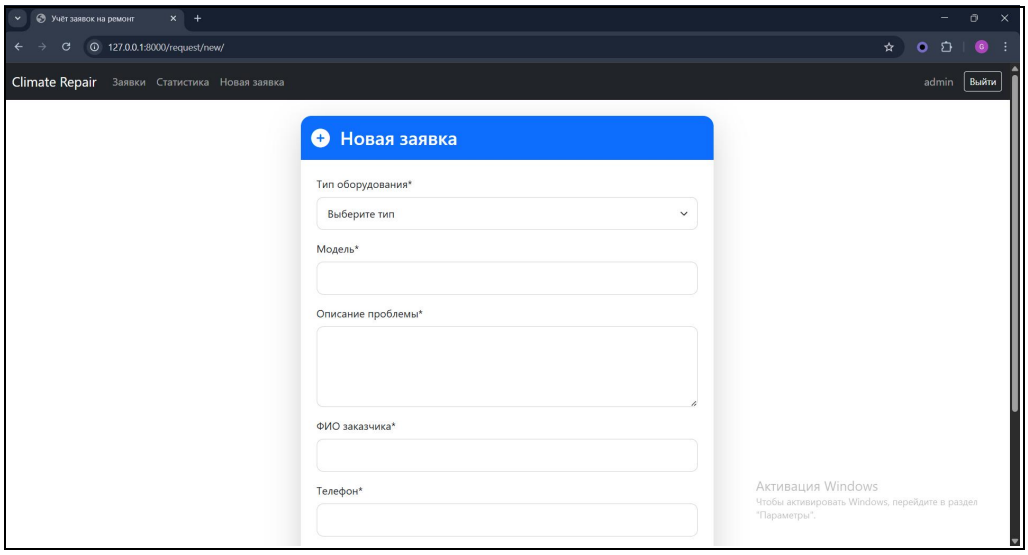


Рисунок 1.7 – Добавление новой заявки

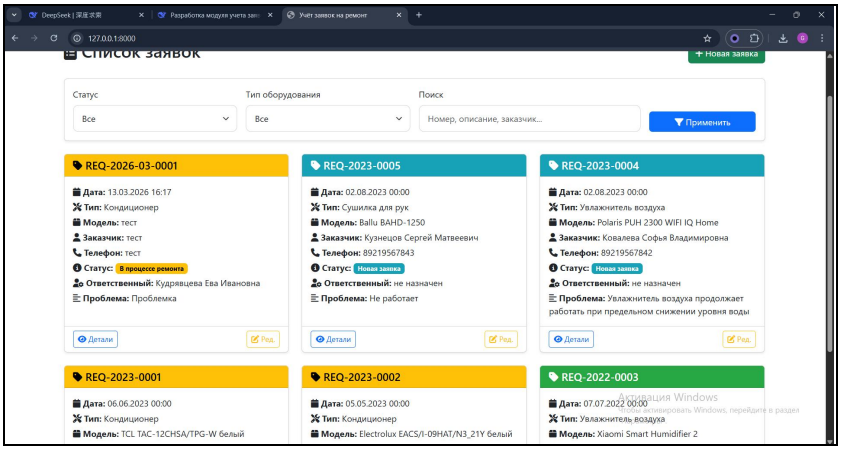


Рисунок 1.8 – Добавленная запись в таблице

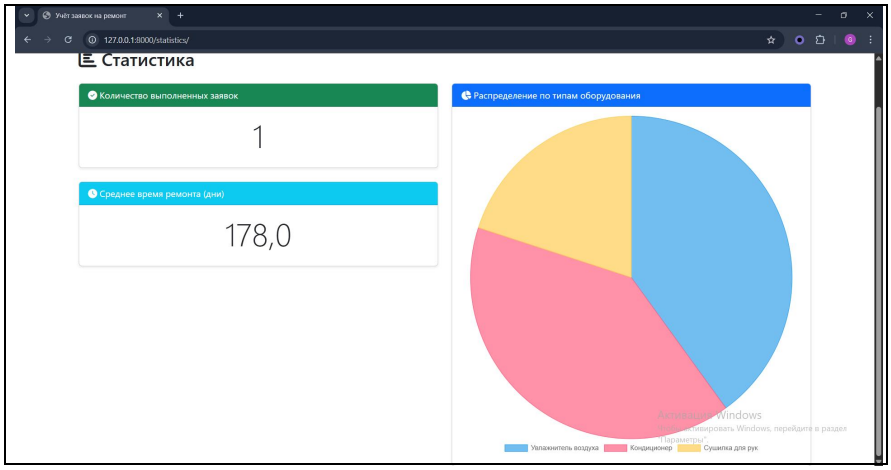


Рисунок 1.9 - Статистика

1.6 Тестирование программы

После реализации программы она была протестирована, тесты и их результаты представлены в таблице 1.1.

Таблица 1.1 – Тестирование программы

№	Проверяемая функция	Входные данные	Ожидаемый результат	Фактический результат	Статус
1	Добавление заявки	Корректные данные (тип, модель, проблема, ФИО, телефон)	Запись сохранена в БД, добавлена в список	Сохранено, отображается в списке	Успешно
2	Валидация формы	Пустое поле «Описание проблемы»	Блокировка сохранения, вывод ошибки	Сохранение заблокировано	Успешно
3	Редактирование	Изменение статуса на «в процессе	Статус обновлен в БД и интерфейсе	Статус успешно обновлен	Успешно

№	Проверяемая функция	Входные данные	Ожидаемый результат	Фактический результат	Статус
		ремонта»			
4	Удаление заявки	Нажатие кнопки «Удалить»	Появление запроса на подтверждение	Окно подтверждения отображается	Успешно
5	Расчет статистики	Переход в раздел (при 3 выполненных заявках в БД)	Вывод числа «3» в поле выполненных заявок	Подсчет совпадает с данными БД	Успешно

По результатам тестирование, все тестирование прошли успешно.

1.7 Руководство системному программисту

1.7.1 Общие сведения о программе

Программа «Учёт заявок на ремонт климатического оборудования» представляет собой веб-приложение, разработанное на языке Python с использованием фреймворка Django. Система предназначена для автоматизации процессов регистрации, обработки и контроля выполнения заявок на ремонт климатического оборудования (кондиционеры, вентиляция, увлажнители и т.п.). Приложение обеспечивает централизованное хранение данных, распределение задач между специалистами, отслеживание статусов и формирование статистической отчётности.

1.7.2 Функции программы

Основные функции программы:

- регистрация новых заявок (оператором или через интерфейс);
- хранение полной информации о заявке: тип и модель оборудования, описание неисправности, контактные данные заказчика, статус выполнения;
- назначение ответственного специалиста по каждой заявке;
- изменение статуса заявки (открыта, в процессе ремонта, ожидание комплектующих, завершена);
- ведение комментариев по заявке (специалисты могут добавлять заметки и информацию о заказанных комплектующих);
- просмотр списка заявок с возможностью фильтрации по статусу, типу оборудования и поиска по тексту;

- расчёт статистических показателей: количество выполненных заявок, среднее время ремонта, распределение по типам неисправностей;
- аутентификация пользователей по логину/паролю и разграничение прав доступа на основе ролей (оператор, специалист, менеджер по качеству).

1.7.3 Минимальный состав технических средств

Минимальная конфигурация для эксплуатации:
- персональный компьютер или сервер с архитектурой x86-64;
- процессор с тактовой частотой не ниже 2,0 ГГц (рекомендуется 2 ядра и более);
- оперативная память – не менее 4 ГБ (рекомендуется 8 ГБ);
- свободное место на диске – не менее 2 ГБ (для кода приложения, базы данных и логов);
- сетевое подключение для доступа пользователей через веб-браузер;
- монитор с разрешением не ниже 1280×1024.

1.7.4 Минимальный состав программных средств

- операционная система: Windows 10/11/Server, Linux (Ubuntu 20.04+, CentOS 7+) или macOS;
- интерпретатор Python версии 3.10 или выше;
- СУБД PostgreSQL версии 14 или выше;

веб-сервер (по выбору):
- встроенный сервер разработки Django (для небольших нагрузок или отладки);
- Gunicorn + nginx (для промышленной эксплуатации);
- браузер пользователя: Microsoft Edge, Google Chrome, Mozilla Firefox (актуальные версии).

1.7.5 Требования к персоналу (системному программисту)

Системный программист, сопровождающий программу, должен:
- знать основы администрирования операционной системы (Windows/Linux);

- владеть навыками установки и настройки Python и PostgreSQL;
- понимать структуру Django-проекта (модели, представления, шаблоны, маршрутизация);
- уметь работать с системой управления базами данных PostgreSQL (создание дампов, восстановление, выполнение SQL-запросов);
- разбираться в механизмах аутентификации и авторизации Django (группы, права доступа);
- уметь читать и анализировать журналы ошибок (логи приложения и сервера).

1.8 Структура программы

1.8.1 Сведения о структуре программы

Программа реализована как веб-приложение на базе фреймворка Django и построена по архитектурному шаблону MTV (Model-Template-View). Основные компоненты:
- точка входа и конфигурация – файлы manage.py, climate_repair/settings.py, climate_repair/urls.py;
- слой доступа к данным (модели) – описание структуры таблиц базы данных и связей между ними (файл requests_app/models.py);
- представления (views) – обработка HTTP-запросов, реализация бизнес-логики (файл requests_app/views.py);
- шаблоны (templates) – HTML-страницы с использованием Bootstrap 5 для отображения интерфейса (директория templates/requests_app/);
- формы (forms) – классы для валидации и генерации HTML-форм (requests_app/forms.py);
- статические ресурсы – CSS-стили, изображения, JavaScript (директория static/requests_app/);
- маршрутизация (URLconf) – сопоставление URL-адресов с соответствующими представлениями (requests_app/urls.py и главный climate_repair/urls.py).

1.8.2 Сведения о составных частях программы

- точка входа и конфигурация: manage.py, climate_repair/settings.py, climate_repair/urls.py Запуск приложения, настройка параметров (база данных, установленные приложения, middleware, шаблоны, статика);

- модели (Models): requests_app/models.py Описание сущностей базы данных: EquipmentType, RequestStatus, Specialist, User, Request, Comment, PartOrder, а также связи между ними;

- представления (Views): requests_app/views.py Реализация логики обработки запросов: отображение списка заявок, детальной информации, создание/редактирование, удаление, статистика, аутентификация;

- шаблоны (Templates): requests_app/templates/requests_app/ HTML-страницы с вставками Django-шаблонизатора: base.html (общий каркас), request_list.html, request_detail.html, request_form.html, statistics.html, login.html, qr_code.html;

- формы (Forms): requests_app/forms.py Классы для создания и валидации форм: RequestForm, CommentForm, PartOrderForm;

- статические файлы: static/requests_app/css/custom.css Кастомные CSS-стили;

- маршрутизация (URLs): requests_app/urls.py, climate_repair/urls.py Определение URL-паттернов и привязка к представлениям;

- управляющие команды: requests_app/management/commands/ Кастомные команды для импорта начальных данных: import_users, import_requests, import_comments.

1.8.2 Сведения о связях между составными частями программы

- Маршрутизация (URLconf) связывает входящие HTTP-запросы с соответствующими представлениями (views).

- Представления используют модели для получения/изменения данных через ORM Django.

- Для валидации и обработки пользовательского ввода представления обращаются к формам.

- После выполнения логики представления передают данные в шаблоны (через контекст), которые генерируют итоговый HTML.

- Шаблоны могут включать статические файлы (CSS, JS) для оформления и интерактивности.

- Модели автоматически связываются с таблицами базы данных через механизм миграций Django.

- Управляющие команды используют модели для наполнения базы данными и могут запускаться независимо от веб-интерфейса.

1.8.3 Сведения о связях с другими программами

Программа взаимодействует с СУБД PostgreSQL через драйвер psycopg2 (встроенная поддержка Django ORM). В производственной среде дополнительно может использоваться веб-сервер nginx для проксирования запросов к Gunicorn и раздачи статических файлов. Встроенный сервер Django предназначен только для разработки. Внешние интеграции (например, отправка уведомлений по электронной почте) в базовой версии не реализованы, но могут быть добавлены при необходимости.

1.9 Настройка программы

1.9.1 Настройка на состав технических средств

- Убедиться в наличии достаточного объёма оперативной памяти и свободного дискового пространства согласно минимальным требованиям.
- При развёртывании на сервере обеспечить постоянную доступность сервера PostgreSQL (локально или по сети).
- При использовании внешнего веб-сервера (nginx) настроить проксирование запросов на порт, где работает Gunicorn (например, 8000), и настроить раздачу статических файлов.
- Открыть необходимые сетевые порты (например, 80 для HTTP, 443 для HTTPS) в брандмауэре.

1.9.2 Настройка на состав программных средств

Установка Python и зависимостей:
- Установить интерпретатор Python версии 3.10+.
- В корневой директории проекта выполнить:

pip install -r requirements.txt

Установка и настройка PostgreSQL:
- Установить СУБД PostgreSQL (например, версию 15).
- Создать базу данных, например, climate_repair:

CREATE DATABASE climate_repair;

- Создать пользователя и предоставить ему права на базу.

Настройка параметров подключения:
- В файле climate_repair/settings.py в словаре DATABASES указать корректные параметры подключения: имя базы, пользователя, пароль, хост и порт.

1.10 Проверка программы

1.10.1 Описание способов проверки

Основные способы проверки корректности работы программы:

1) Функциональное тестирование

- проверка входа/выхода и разграничения прав доступа для различных ролей (оператор, специалист, менеджер);
- проверка создания, редактирования и удаления заявок;
- проверка добавления комментариев и заказов комплектующих;
- проверка поиска и фильтрации заявок по различным критериям;
- проверка отображения статистики (количество выполненных заявок, среднее время ремонта, диаграмма распределения).

2) Тестирование интеграции с базой данных

- проверка подключения к PostgreSQL и выполнения запросов;
- проверка корректности работы первичных и внешних ключей, ограничений целостности;
- проверка автоматического формирования номера заявки и дат создания/завершения;
- проверка обработки ошибок уникальности (например, дублирование номера заявки).

3) Тестирование отказоустойчивости

- поведение при временной недоступности базы данных – приложение должно выдавать понятное сообщение об ошибке, а не падать;
- корректная обработка исключений при сохранении данных (например, IntegrityError);
- проверка логирования ошибок в файл или консоль.

4) Нагрузочное тестирование (опционально)

- оценка времени отклика при работе с большим количеством заявок (например, 10 000 записей);
- проверка стабильности при одновременном доступе нескольких пользователей.

1.11 Сообщения системному программисту

В процессе работы возможны следующие типичные сообщения и ситуации (таблица 1.2):

Таблица 1.2 – Сообщение, причины и решение

Сообщение / ситуация	Возможная причина	Действия системного программиста
Ошибка ModuleNotFoundError: No module named 'requests_app'	Приложение не зарегистрировано в INSTALLED_APPS.	Добавить 'requests_app' в список INSTALLED_APPS файла settings.py.
Ошибка подключения к базе данных	Неверные параметры в DATABASES (имя, пользователь, пароль) или не запущен сервер PostgreSQL.	Проверить корректность данных в settings.py. Убедиться, что сервис PostgreSQL запущен.
Ошибка TemplateDoesNotExist	Отсутствует файл шаблона по указанному пути.	Убедиться, что шаблон лежит в правильной директории (templates/requests_app/) или указан корректный путь в представлении.
Не применяются CSS-стили	Неправильно настроены статические файлы.	Выполнить команду python manage.py collectstatic. Проверить, что в браузере не кэшируется старая версия CSS (выполнить "жесткую" перезагрузку).
Проблемы с правами доступа	Пользователь не имеет нужной роли или обращается к чужим данным	Зайти в админ-панель Django (/admin) и проверить/назначить группы и разрешения для пользователя.

2 Дипломный проект

2.1 Введение дипломного проекта

В условиях цифровой трансформации государственного управления и системы образования одной из ключевых задач является оптимизация процессов сбора, обработки и представления статистической информации. Ежегодная форма федерального статистического наблюдения ОШ-1 «Сведения об организации, осуществляющей образовательную деятельность по образовательным программам начального общего, основного общего, среднего общего образования» является основным инструментом для формирования полной и достоверной картины состояния общего образования в Российской Федерации. Однако традиционный процесс её заполнения, основанный на использовании локальных табличных редакторов и ручном переносе данных, остается трудоемким, подверженным ошибкам и требует значительных временных затрат от ответственных сотрудников образовательных организаций.

Создание специализированного программного приложения, автоматизирующего процесс заполнения, проверки и отправки формы ОШ-1, позволит существенно повысить эффективность работы, минимизировать арифметические и логические ошибки, обеспечить стандартизацию данных и снизить административную нагрузку на учреждения. Таким образом, разработка такого приложения представляет собой актуальную практическую задачу, направленную на решение конкретных проблем в сфере образовательной статистики.

Цель курсового проекта – разработать функциональный прототип веб-приложения для автоматизированного формирования и проверки статистической отчетности по форме ОШ-1, обеспечивающего интуитивно понятный интерфейс ввода, валидацию данных и экспорт результата в установленный формат.

Задачи курсового проекта:

1) провести анализ предметной области: изучить структуру, показатели, правила заполнения и актуальные требования к форме федерального статистического наблюдения ОШ-1.

2) сформулировать функциональные и нефункциональные требования к проектируемому приложению на основе выявленных потребностей пользователей.

3) спроектировать архитектуру и логику приложения: выбрать технологический стек (Frontend и Backend), определить структуру базы данных, разработать алгоритмы расчетов производных показателей и логику проверок на корректность данных.

4) реализовать основной функционал приложения, включая:

- модуль авторизации и разграничения прав доступа;
- интерактивный интерфейс для пошагового ввода данных с понятной структурой, повторяющей логику формы;

- модуль автоматической валидации введенных данных (проверка на целостность, соответствие контрольным соотношениям, арифметическая правильность);

- систему уведомлений об ошибках и подсказок для пользователя.

5) провести тестирование работоспособности ключевых функций прототипа и его использование.

2.2 Анализ предметной области

Средняя общеобразовательная школа № 1 в Первоуральске находится по адресу ул. Строителей, 7. Это муниципальное автономное общеобразовательное учреждение (МАОУ СОШ № 1), которое работает с 1995 года и предлагает программы начального, основного и среднего общего образования. В школе также есть кружки, секции, проектная деятельность и психологическая поддержка. Директором школы с 2022 года является Забродина Ольга Алексеевна.

Структура МАОУ СОШ №1 города Первоуральска включает следующие ключевые элементы:

1.Учредитель, отвечает за управление образования администрации городского округа Первоуральск.

2.Руководство

- директор;
- заместители директора;

3.Педагогический состав

- учителя начальных классов, основного общего и среднего общего образования, а также другие педагогические работники (учителя-предметники, психологи, социальные педагоги и т.д.).

4.Административно-хозяйственный персонал.

5.Образовательные уровни (структурные подразделения)

- начальное общее образование;
- основное общее образование;
- среднее общее образование;

В настоящее время процесс подготовки и сдачи статистической отчетности по форме ОШ-1 сопряжен со значительными трудностями. Данные, необходимые для заполнения отчета, часто хранятся в разрозненных системах и документах (бухгалтерских регистрах, отделах кадров, производственных отчетах). Сотрудники вынуждены вручную собирать информацию, консолидировать ее и переносить в установленный шаблон, что является крайне трудоемким и медленным процессом. Высокая доля ручного труда многократно увеличивает риск возникновения арифметических и логических ошибок, опечаток и несоответствий.

Разработка специализированного программного модуля для подготовки статистической отчетности по форме ОШ-1 позволит комплексно решить существующие проблемы.

Ключевым преимуществом является возможность автоматического формирования готового отчета ОШ-1 на основе утвержденных алгоритмов и правил. Это позволит сотрудникам не тратить время на рутинный сбор и ввод данных, а сосредоточиться на контроле и проверке информации. Система сможет самостоятельно проводить базовые проверки на целостность данных, арифметическую и логическую согласованность показателей, минимизируя человеческий фактор.

Внедрение разрабатываемого программного модуля позволит решить указанные проблемы за счет автоматизации сбора данных, централизованного хранения показателей и исключения ручного переноса информации. Это повысит скорость и точность подготовки отчета ОШ-1, снизит нагрузку на персон

2.3 Требование к своему программному продукту

Программа предназначена для помощи учета сведений об организации, осуществляющей образовательную деятельность по программам начального общего, основного общего и среднего общего образования.

Функциональные требования должны предоставлять авторизованным пользователям организации возможность полного цикла работы со статистической отчетностью по форме ОШ-1. Система начинается с разграничения прав доступа, предоставляя уникальные интерфейсы и возможности для разных ролей: администратора, бухгалтера, ответственного статистика и руководителя. Каждый пользователь работает в своей зоне ответственности, а система фиксирует все значимые действия в журнале аудита.

Ключевой функцией является предоставление интерактивной электронной формы, в точности повторяющей актуальную официальную форму ОШ-1. В процессе заполнения система обязана проводить непрерывную валидацию введенных данных, проверяя типы значений, арифметическую корректность (сходимость сумм, баланс показателей) и соответствие утвержденным справочникам (ОКПО, ОКВЭД и др.). Для упрощения массового ввода данных должна быть реализована возможность импорта из внешних файлов, таких как XLSX или CSV, особенно для табличных разделов о персонале и контингенте учащихся. Система должна позволять сохранять незавершенную работу как черновик и поддерживать консолидацию данных из нескольких филиалов в единый отчет по организации.

Перед окончательным формированием отчета пользователь должен иметь возможность запустить комплексную проверку, по результатам которой система генерирует детальный протокол с указанием ошибок, требующих исправления, и предупреждений. Все расчетные показатели (средние значения, проценты, доли)

рассчитываются автоматически на основе введенных первичных данных. Важной вспомогательной функцией является анализ динамики путем сравнения с данными прошлого отчетного периода для выявления аномальных отклонений.

После устранения всех замечаний система формирует итоговый пакет отчетности. Она должна генерировать файл в формате XML, соответствующий утвержденной XSD-схеме Росстата, для последующей электронной отправки, а также создавать печатную версию отчета в формате PDF для внутреннего архивирования. Дополнительно предусматривается выгрузка данных в форматах Excel и Word для служебного использования.

Финальным этапом является отправка отчета в контролирующий орган. Система должна обеспечивать интеграцию с официальными шлюзами приема отчетности (например, через СМЭВ) для прямого взаимодействия. Отправка осуществляется только с использованием усиленной квалифицированной электронной подписи уполномоченного лица. После отправки система обязана получать, сохранять и отображать пользователю входящую квитанцию о приеме отчета от Росстата. Все статусы отчета — от создания черновика до официального приема — фиксируются в едином журнале отправок.

Для поддержки пользователей система должна быть оснащена встроенными актуальными справочниками, контекстной справкой по полям формы и лентой новостей об изменениях в методологии или законодательстве.

Нефункциональные требования требуют, чтобы программный продукт обладал высоким уровнем надежности и доступности. В рабочее время (с 8:00 до 20:00 по московскому времени) доступность системы должна составлять не менее 99.5%. Для обеспечения сохранности данных необходимо выполнять автоматическое ежедневное резервное копирование с хранением копий не менее 30 дней. В случае возникновения инцидента время на восстановление работоспособности системы не должно превышать 4 часов, а максимальная возможная потеря данных должна ограничиваться одним часом.

Безявляется критически важным аспектом. Программный продукт обязан полностью соответствовать требованиям Федерального закона №152-ФЗ «О персональных данных», что подразумевает необходимость соответствующей сертификации или аудита. Все передаваемые данные должны защищаться с использованием протокола TLS версии не ниже 1.2. Пароли пользователей хранятся исключительно в хэшированном виде. Система должна быть защищена от распространенных веб-атак, таких как SQL-инъекции, межсайтовый скриптинг (XSS) и подделка межсайтовых запросов (CSRF). Конфиденциальная информация должна шифроваться не только при передаче, но и при хранении. Журналы аудита, содержащие полную историю действий пользователей, подлежат хранению в течение как минимум 3 лет.

Интерфейс системы должен быть ориентирован на удобство пользователя с разным уровнем компьютерной грамотности. Он должен быть интуитивно понятным, не требующим длительного обучения, и корректно отображаться как на персональных компьютерах, так и на планшетных устройствах. Наличие подробной документации и наглядных видеоинструкций является обязательным.

Система должна демонстрировать высокую степень совместимости. Она должна корректно функционировать в последних версиях основных браузеров, включая Chrome, Firefox, Edge и Yandex Browser. Необходима поддержка работы с аппаратными и программными средствами криптографической защиты информации (электронная подпись), выпущенными аккредитованными удостоверяющими центрами.

Архитектура системы должна обеспечивать легкость ее сопровождения и развития. Обновление нормативно-справочной информации (форм, кодов, классификаторов) должно осуществляться администратором через специальный интерфейс без необходимости вмешательства в программный код. Система должна быть построена по модульному принципу, позволяющему в будущем относительно просто добавлять поддержку новых форм статистической отчетности.

Наконец, продукт должен строго соответствовать всем юридическим и регуляторным нормативам. Это включает в себя не только точное следование официальной методологии заполнения формы ОШ-1, но и обеспечение полной юридической значимости создаваемых и отправляемых электронных документов за счет соблюдения установленного порядка представления отчетности и использования корректных средств электронной подписи.

Эксплуатационные требования требуют, чтобы программный продукт был спроектирован и реализован с учетом удобства и безопасности его постоянной эксплуатации в реальных условиях образовательной организации или уполномоченного оператора. Требования к эксплуатации охватывают процессы развертывания, поддержки, обновления и мониторинга системы на протяжении всего ее жизненного цикла.

Процедура обновления является критически важной. Обновления системы, включающие исправления ошибок, обновления справочников и методологии, а также добавление нового функционала, должны применяться централизованно для облачной версии с минимальным временем простоя, согласованным с клиентами в регламенте техобслуживания. Для локальной версии должен предоставляться подробный регламент обновления, предпочтительно в виде автоматизированных скриптов. Все обновления должны быть обратимы; должна существовать четкая и проверенная процедура отката к предыдущей стабильной версии в случае возникновения критических проблем после обновления. Обновление нормативно-справочной информации (форм отчетности, кодов) должно, по возможности, производиться «на лету», без перезапуска приложения.

Наконец, система должна быть масштабируемой. Должны быть четко указаны рекомендации по масштабированию аппаратных ресурсов при росте объема хранимых данных. Процедуры аварийного восстановления (Disaster Recovery), включая восстановление из резервной копии на резервной площадке, должны быть четко формализованы и регулярно тестироваться.

Аппаратные требования к серверу, для запуска веб-приложения

- процессор: 2 ядра, 2.4 GHz;
- оперативная память: 4 GB;

- жёсткий диск: 20 GB;
- веб-браузер последней версии;
- доступ в Интернет.

Данные требования являются минимальными и обеспечивают базовую функциональность системы.

2.4 Обоснование выбора инструментальных средств разработки

Для разробатки данного веб-приложения был выбран следующий технологический стек:

- backend: Python;
- frontend: JavaScript, CSS, HTML;
- база данных: PostgreSQL.

В качестве основного языка серверной разработки выбран Python с фреймворком Django. Python — мультипарадигменный высокоуровневый язык программирования общего назначения с динамической строгой типизацией и автоматическим управлением памятью, ориентированный на повышение производительности разработчика, читаемости кода и его качества, а также на обеспечение переносимости написанных на нём программ.

Для реализации клиентской части применен стандартный для веб-разработки набор технологий: HTML, CSS и JavaScript. Использование современных стандартов HTML5 и CSS3 обеспечивает создание структурированного, семантически верного и адаптивного пользовательского интерфейса, который корректно работает на различных устройствах, включая планшеты. JavaScript является основой для создания динамического, интерактивного интерфейса, который должен реагировать на действия пользователя без постоянной перезагрузки страницы (например, валидация полей на лету, динамическое добавление строк в таблицы, асинхронная отправка данных).

В качестве системы управления базами данных выбрана PostgreSQL. Это одна из самых передовых и надежных реляционных СУБД с открытым исходным кодом, что полностью соответствует строгим требованиям проекта к целостности и безопасности данных.

Выбор данных инструментов разработки был обусловлен просьбой заказчика.

2.5 Разработка архитектуры блок-схем программы

В данном разделе представлена блок-схема, отражающая общий алгоритм функционирования разработанного веб-приложения, предназначенного для сдачи

статистической отчетности ОШ-1. Использование блок-схемы позволяет визуально представить последовательность операций и взаимодействие основных модулей системы, что упрощает понимание логики работы приложения и служит основой для его дальнейшей программной реализации.

Блок-схема отражает взаимодействие этих компонентов в рамках единого алгоритма обработки данных. В ней последовательно представлены этапы получения входной информации, ее валидации, выполнения соответствующих действий и формирования результата, возвращаемого пользователю.

На рисунке 2.1 приведена разработанная архитектура блок-схемы веб-приложения.

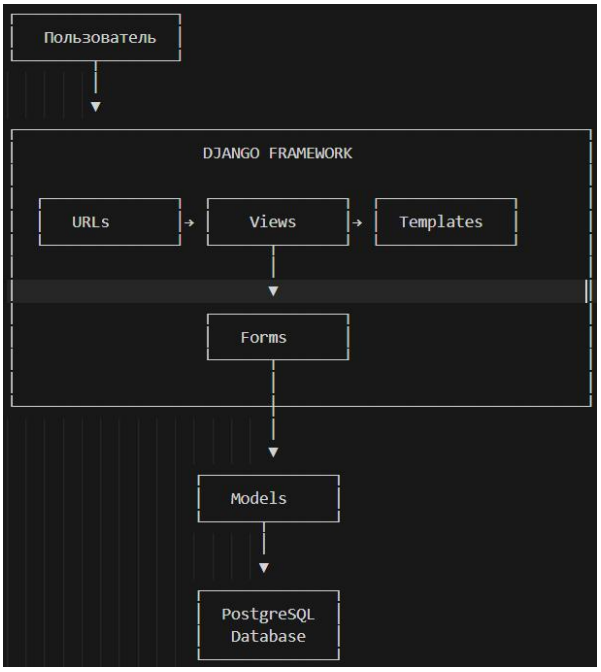


Рисунок 2.1 – Схема архитектуры веб-приложения

2.6 Разработка макетов основных форм приложения

На этапе проектирования веб-приложения важной задачей является разработка макетов пользовательских форм, которые определяют внешний вид интерфейса и структуру взаимодействия пользователя с системой. Макеты служат прототипами будущих страниц и позволяют заранее продумать логику размещения элементов управления, механизмы ввода данных и способы отображения информации. Такой подход позволяет упростить последующую реализацию интерфейса, снизить вероятность ошибок и обеспечить удобство работы операторов.

При создании макетов учитывались особенности предметной области – необходимость быстрого ввода данных, точность фиксации показателей и минимизация количества действий при выполнении операций. Поэтому при

разработке форм использовались принципы простоты, последовательности и интуитивной понятности. Все формы были разделены на несколько функциональных групп в соответствии с задачами системы.

Основные макеты включают:

1) форму авторизации пользователя, предназначенную для ввода логина и пароля. Макет содержит минимальный набор элементов: поля ввода, кнопку входа и ссылку для регистрации. Структура формы выполнена так, чтобы обеспечить быстрый доступ к системе (Рисунок 2.2).

Рисунок 2.2 – Макет авторизации

2) главную страницу (панель навигации), обеспечивающую переход ко всем ключевым разделам приложения. В макете предусмотрено боковое меню, содержащее пункты. Такой подход обеспечивает удобную навигацию и упрощает ориентацию пользователя в интерфейсе (Рисунок 2.3).

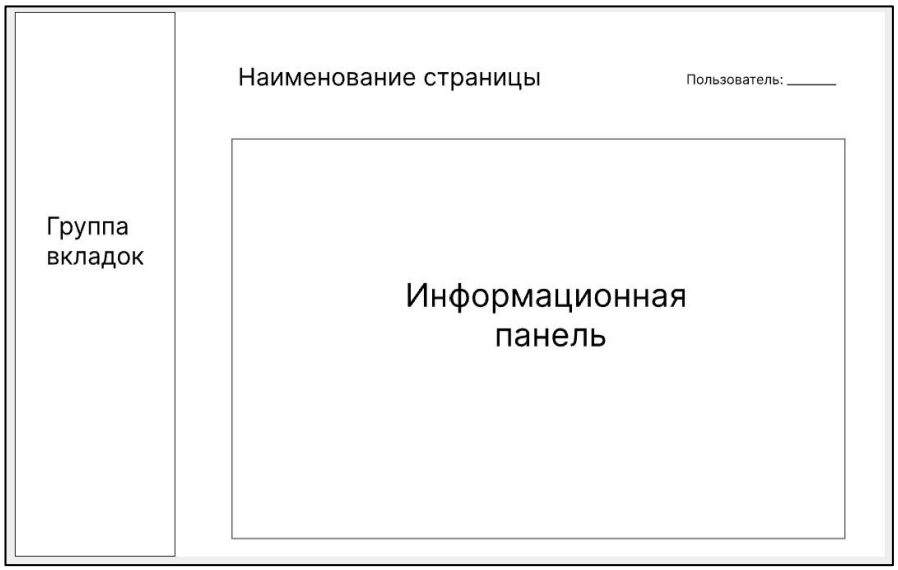


Рисунок 2.3 – Макет главной страницы

3) форму редактирова отчета, включающую таблицу с некоторыми обязательными параметрами. Макет предусматривает возможность добавления, редактирования и сохранения записей в базе данных (Рисунок 2.4).

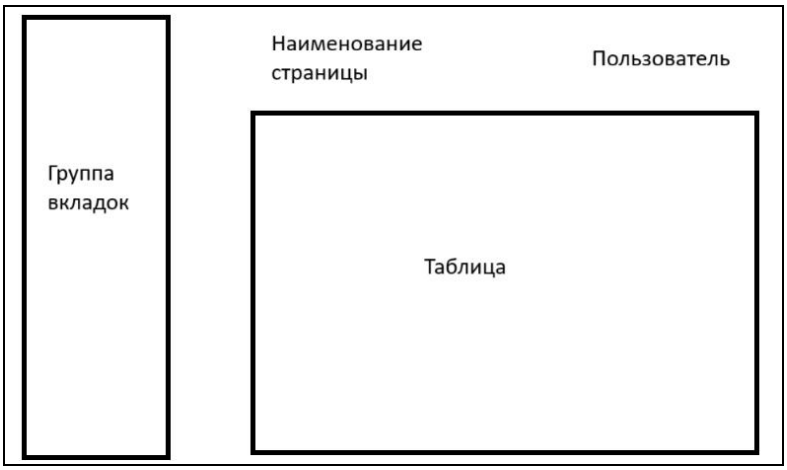


Рисунок 2.4 – Макет страницы для редактирования отчета

4) форму содержащую данные об учениках, предназначенную для заполнения отчета (Рисунок 2.5).

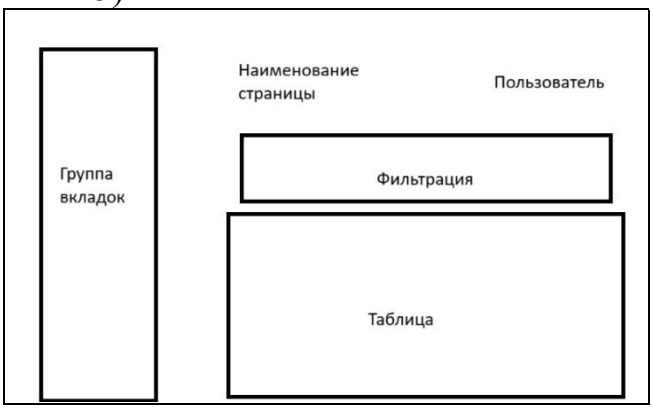


Рисунок 2.5 – Макет страницы регистрации операций

Каждый макет проектировался с учетом принципов адаптивности, что позволяет корректно отображать страницы как на настольных компьютерах, так и на мобильных устройствах. На основе разработанных макетов была выполнена последующая реализация интерфейса веб-приложения с использованием современных веб-технологий.

2.7 Проектирование и разработка базы данных

Проектирование базы данных является одним из ключевых этапов разработки информационной системы для сдачи отчета ОШ-1. Целью данного

этапа является создание структуры хранения данных, которая обеспечивает корректное отображение данных.

На основе анализа предметной области были определены основная сущность и взаимосвязи, характерные для создания отчета. В качестве базового элемента были выделены: ученики. Эта сущности образует фундаментальную логику системы и обеспечивает автоматизацию формирования отчета.

Структура базы данных включает следующие основные таблицы:

1) auth_user – пользователи системы;
– id: serial primary key;
– password: varchar(128);
– last_login: timestamp;
– is_superuser: boolean;
– username: varchar(150) unique;
– first_name: varchar(150);
– last_name: varchar(150);
– email: varchar(254);
– is_staff: boolean;
– is_active: boolean;
– date_joined: timestamp;

2)osh1_educationalinstitution – образовательные организации;
– id: serial primary key;
– name: varchar(255);
– region: varchar(120);
– district: varchar(120);
– address: varchar(255);
– inn: varchar(12) unique;
– director: varchar(255);
– email: varchar(254);

3) osh1_reportperiod – отчётные периоды;
– id: serial primary key;
– year: integer;
– quarter: smallint;
– deadline: date;
– unique(year, quarter);

4) osh1_oshreport – отчёты ОШ-1;
– id: serial primary key;
– institution_id: integer foreign key (osh1_educationalinstitution.id);
– period_id: integer foreign key (osh1_reportperiod.id);
– status: varchar(20);
– submitted_at: timestamp;
– submitted_by: varchar(255);
– contact_phone: varchar(32);
– total_students: integer;
– female_students: integer;

– boarding_students: integer;
– graduating_students: integer;
– graduates_entered_university: integer;
– teachers_total: integer;
– teachers_high_category: integer;
– comments: text;
– attachment: varchar(100);
– unique(institution_id, period_id);
5) osh1_section2indicator – справочник строк раздела 2;
– id: serial primary key;
– row_code: varchar(4);
– name: varchar(512);
– order: smallint;
6) osh1_section2value – значения строк раздела 2 для отчётов;
– id: serial primary key;
– report_id: integer foreign key (osh1_oshreport.id);
– indicator_id: integer foreign key (osh1_section2indicator.id);
– total: integer;
– grade_1: integer;
– grade_2: integer;
– grade_3: integer;
– grade_4: integer;
– grade_5: integer;
– grade_6: integer;
– grade_7: integer;
– grade_8: integer;
– grade_9: integer;
– grade_10: integer;
– grade_11: integer;
– grade_11_grad: integer;
– grade_12: integer;
– unique(report_id, indicator_id);
7) osh1_student – обучающиеся;
– id: serial primary key;
– report_id: integer foreign key (osh1_oshreport.id);
– full_name: varchar(255);
– gender: varchar(1);
– education_level: varchar(16);
– grade: smallint;
– age: smallint;
– class_type: varchar(16);
– has_ovz: boolean;
– is_disabled: boolean;
– is_orphan: boolean;

– without_parental_care: boolean;
– movement_status: varchar(8);
– form_of_education: varchar(16);
– is_extern: boolean;

Между таблицами установлены связи через первичные и внешние ключи. Все таблицы приведены к нормальным формам, что позволяет избежать избыточности, предотвратить дублирование информации и повысить стабильность работы системы. Внешние ключи обеспечивают защиту от некорректного удаления или изменения связанных данных.

При физической реализации модели в СУБД PostgreSQL были выбраны оптимальные типы данных (integer, varchar, date, text). На уровне базы данных реализованы механизмы автоматического заполнения базы данных с помощью триггеров, что обеспечивает корректность данных при любом изменении связанных документов. Структура БД была разработана таким образом, чтобы допускать дальнейшее расширение и включение дополнительных модулей отчёта, не нарушая существующую архитектуру.

Ниже на рисунке 2.5 представлена ERD-диаграмма, отражающая структуру базы данных и основные взаимосвязи между сущностями.

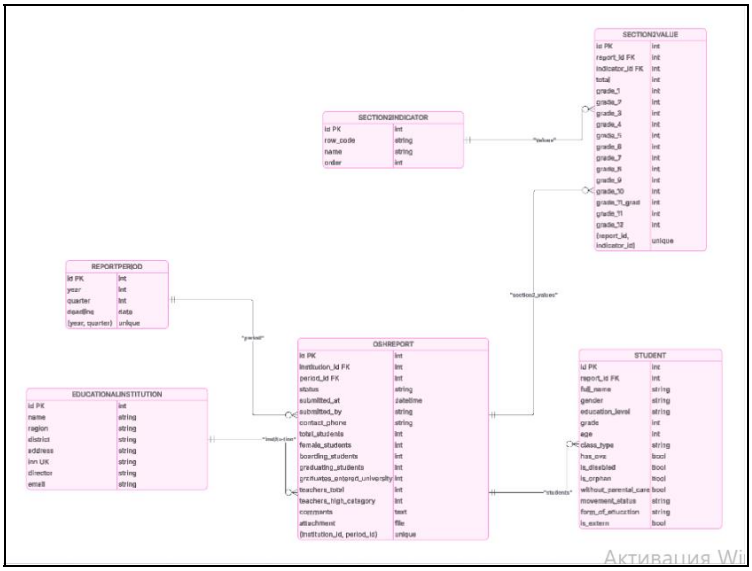


Рисунок 2.5 – ERD-диаграмма БД

2.8 Разработка пользовательского интерфейса

Страница «Панель управления». Пользовательский интерфейс веб-приложения состоит из двух основных областей: боковой панели навигации и основной рабочей области. Боковая панель содержит меню выбора разделов системы. Основная область включает заголовок, ближайший дедлайн и таблицу с последними отчетами (Рисунок 2.6).

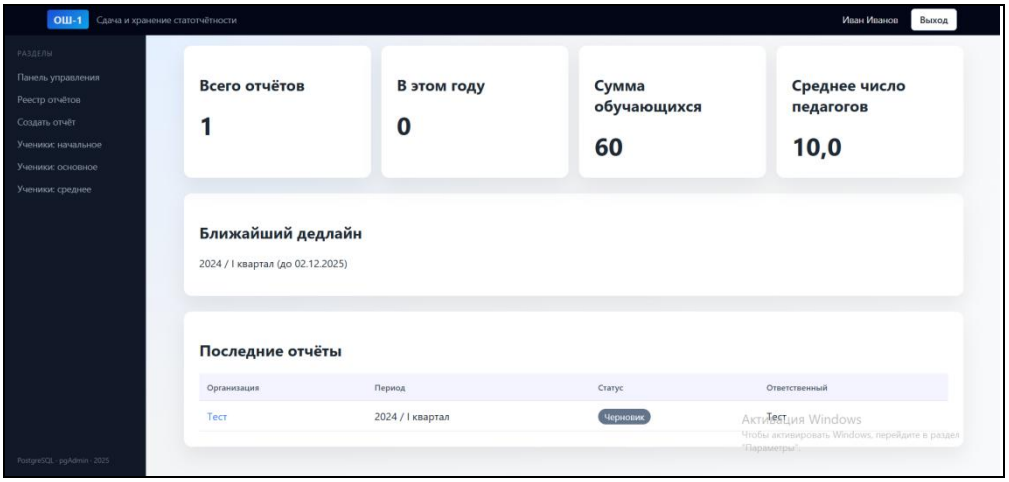


Рисунок 2.6 – Общий вид интерфейса приложения

HTML-структура основного контейнера и таблицы с заголовками колонок «Организация», «Период», «Статус», «Ответственный» представлена в листинге 2.1.

Листинг 2.1 – HTML-структура основного контейнера и таблицы

```
{% extends "base.html" %}

{% block title %}Панель мониторинга{% endblock %}

{% block content %}
<section class="grid">
    <article class="card">
        <h2>Всего отчётов</h2>
        <p class="metric">{{ reports_total }}</p>
    </article>
    <article class="card">
        <h2>В этом году</h2>
        <p class="metric">{{ reports_current_year }}</p>
    </article>
    <article class="card">
        <h2>Сумма обучающихся</h2>
        <p class="metric">{{ students_total }}</p>
    </article>
    <article class="card">
        <h2>Среднее число педагогов</h2>
<p class="metric">{{ teachers_avg }}</p>
    </article>
</section>

<section class="card mt-lg">
    <h2>Ближайший дедлайн</h2>
    {% if next_deadline %}
        <p>{{ next_deadline }} (до
{{ next_deadline.deadline|date:"d.m.Y" }})</p>
```

```
        {% else %}
            <p>Пока нет активных периодов.</p>
        {% endif %}
    </section>

    <section class="card mt-lg">
        <h2>Последние отчёты</h2>
        <table>
            <thead>
            <tr>
                <th>Организация</th>
                <th>Период</th>
                <th>Статус</th>
                <th>Ответственный</th>
            </tr>
            </thead>
            <tbody>
            {% for report in latest_reports %}
                <tr>
                    <td><a href="{% url 'report_detail'
report.pk %}">{{ report.institution }}</a></td>
                    <td>{{ report.period }}</td>
                    <td><span class="badge badge-
{{ report.status }}">{{ report.get_status_display }}</span></td>
                    <td>{{ report.submitted_by }}</td>
                </tr>
            {% empty %}
                <tr>
                    <td colspan="4">Отчётов пока нет.</td>
                </tr>
            {% endfor %}
            </tbody>
        </table>
    </section>
    {% endblock %}
```

Для визуального оформления элементов интерфейса использовались каскадные таблицы стилей (CSS). Таблица данных имеет скругленные углы и эффект тени, что улучшает ее восприятие. Для повышения удобства работы реализовано подсвечивание строк таблицы при наведении курсора мыши. Основные стилевые правила представлены в листинге 2.2.

Листинг 2.2 – Ключевые CSS-стили для таблицы и модального окна

```
    :root {
        --accent: #3b82f6;
        --accent-soft: rgba(59, 130, 246, 0.12);
        --bg: #f3f4f6;
        --bg-elevated: #ffffff;
        --text: #1f2933;
        --muted: #6b7280;
```

```
        --border-subtle: #e5e7eb;
        --sidebar-bg: #111827;
        --sidebar-border: #1f2937;
        --sidebar-link: #9ca3af;
        --sidebar-link-active-bg: rgba(59, 130, 246, 0.12);
        --sidebar-link-active: #e5e7eb;
    }

    * {
        box-sizing: border-box;
    }

    body {
        font-family: "Segoe UI", -apple-system, BlinkMacSystemFont,
"Roboto", sans-serif;
        background: var(--bg);
        margin: 0;
        color: var(--text);
    }
    a {
        color: var(--accent);
        text-decoration: none;
    }
```

Основная логика работы реализована на Python. При загрузке страницы данные загружаются с сервера через API и отображаются в таблице. Код функции загрузки данных показан в листинге 2.3.

Листинг 2.3 – Функция загрузки данных с сервера

```
    class ReportListView(LoginRequiredMixin, ListView):
        model = OshReport
        template_name = "osh1/report_list.html"
        context_object_name = "reports"
        paginate_by = 10

        def get_queryset(self):
            queryset = (
                OshReport.objects.select_related("institution",
"period")
                .order_by("-submitted_at")
                .all()
            )
            self.filter_form = ReportFilterForm(self.request.GET
or None)
            if self.filter_form.is_valid():
                queryset =
self.filter_form.filter_queryset(queryset)
            return queryset

        def get_context_data(self, **kwargs):
            context = super().get_context_data(**kwargs)
```

```
            context["filter_form"] = getattr(self, "filter_form",
ReportFilterForm())
    return context
```

Для добавления записей используется модальное окно с формой ввода данных. Внешний вид модального окна добавления записи показан на рисунке 2.7. При нажатии на кнопку «Создать отчет» открывается пустая таблица для отчета, показанная на рисунке 2.8. Работа с модальным окном и таблицей, в том числе редактированием и удалением показаны в листинге 2.4..

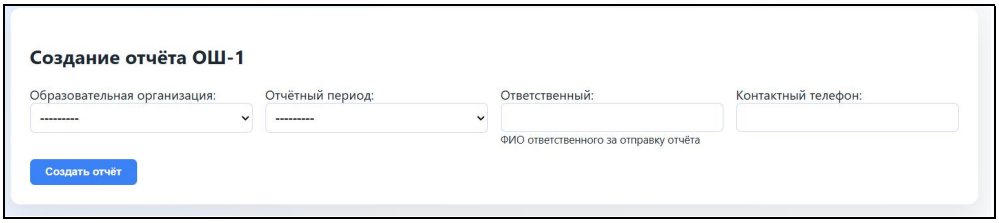


Рисунок 2.7 – Модальное окно добавления отчета

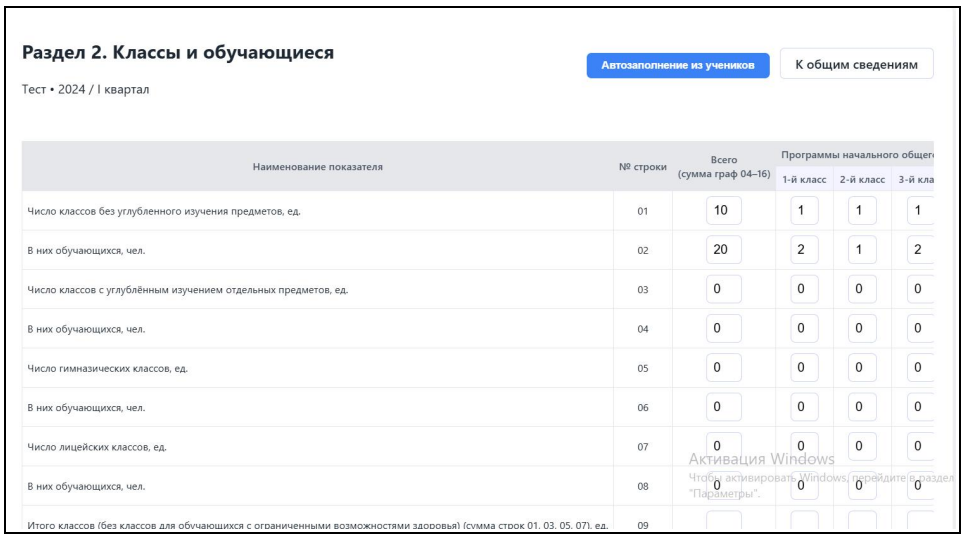


Рисунок 2.8 - Раздел с отчетом

Листинг 2.4 – Работа с модальным окном и таблицей

```
@login_required
def report_create(request):
    from django.forms import modelformset_factory

    report = None
    form = ReportCreateForm(request.POST or None)
    formset = None

    if request.method == "POST":
        if "create_report" in request.POST:
            # Создание отчёта
            if form.is_valid():
                report = form.save(commit=False)
                report.status = OshReport.STATUS_DRAFT
```

```
                        report.save()
                        # Создаём пустые записи для раздела 2
                        indicators =
Section2Indicator.objects.all()
                        for indicator in indicators:

Section2Value.objects.get_or_create(report=report,
indicator=indicator)
                        messages.success(request, "Отчёт создан.
Заполните раздел 2.")
                        # Перезагружаем страницу с созданным
отчётом
                        return
redirect(f"{reverse('report_create')}?report_id={report.pk}")
              elif "update_report" in request.POST and
"report_id" in request.POST:
                  # Обновление отчёта
                  report = get_object_or_404(OshReport,
pk=request.POST["report_id"])
                  form = ReportCreateForm(request.POST,
instance=report)
           if form.is_valid():
      form.save()
                        messages.success(request, "Отчёт
обновлён.")
                        return
redirect(f"{reverse('report_create')}?report_id={report.pk}")
              elif "save_section2" in request.POST and
"report_id" in request.POST:
                  # Сохранение раздела 2
                  report = get_object_or_404(OshReport,
pk=request.POST["report_id"])
                  indicators = Section2Indicator.objects.all()
                  existing =
Section2Value.objects.filter(report=report)
                  existing_map = {value.indicator_id: value
for value in existing.select_related("indicator")}
                  new_values = []
                  for indicator in indicators:
                      if indicator.id not in existing_map:

new_values.append(Section2Value(report=report,
indicator=indicator))
                  if new_values:

Section2Value.objects.bulk_create(new_values)
                  queryset = (

Section2Value.objects.filter(report=report)
                        .select_related("indicator")
                        .order_by("indicator__order",
"indicator__row_code")
```

```
                    )
                    FormSet = modelformset_factory(Section2Value,
form=Section2ValueForm, extra=0, can_delete=False)
                    formset = FormSet(request.POST,
queryset=queryset)
                    if formset.is_valid():
                        formset.save()
                        messages.success(request, "Раздел 2
успешно сохранён.")
                        return redirect("report_detail",
pk=report.pk)
                elif "auto_fill" in request.POST and "report_id"
in request.POST:
                    # Автозаполнение раздела 2
                    report = get_object_or_404(OshReport,
pk=request.POST["report_id"])
                    auto_fill_section2(report)
                    messages.success(request, "Раздел 2
автоматически заполнен на основе данных об учениках.")
                    return
redirect(f"{reverse('report_create')}?report_id={report.pk}")

            # Если отчёт уже создан (из GET параметра),
показываем форму редактирования + раздел 2
            if not report and request.method == "GET":
                report_id = request.GET.get("report_id")
                if report_id:
                    try:
                        report =
OshReport.objects.get(pk=report_id)
                    except (OshReport.DoesNotExist, ValueError):
                        pass

            if report:
                # Показываем форму редактирования отчёта и
таблицу раздела 2
                if request.method == "GET" or "update_report"
not in request.POST:
                    form = ReportCreateForm(instance=report)
                indicators = Section2Indicator.objects.all()
                existing =
Section2Value.objects.filter(report=report)
                existing_map = {value.indicator_id: value for
value in existing.select_related("indicator")}
                new_values = []
                for indicator in indicators:
                    if indicator.id not in existing_map:

new_values.append(Section2Value(report=report,
indicator=indicator))
                if new_values:
                    Section2Value.objects.bulk_create(new_values)
```

```
            queryset = (
                Section2Value.objects.filter(report=report)
                .select_related("indicator")
                .order_by("indicator__order",
"indicator__row_code")
            )
            FormSet = modelformset_factory(Section2Value,
form=Section2ValueForm, extra=0, can_delete=False)
            formset = FormSet(queryset=queryset)
        return render(
            request,
            "osh1/report_form.html",
            {
                "form": form,
                "report": report,
                "formset": formset,
            },
        )
```

Интерфейс обеспечивает полный цикл работы с отчетом: просмотр, добавление, редактирование, удаление, поиск и сортировку. Все операции выполняются без перезагрузки страницы, что обеспечивает удобство и эффективность работы с системой.

Страница «обучающиеся».

Интерфейс страницы об учениках состоит из боковой панели навигации и основной рабочей области. Боковая панель содержит многоуровневое меню с разделами системы, а основная область включает таблицу содержащую информаицю об учениках и фильтрацию. Общий вид интерфейса представлен на рисунке 2.9.

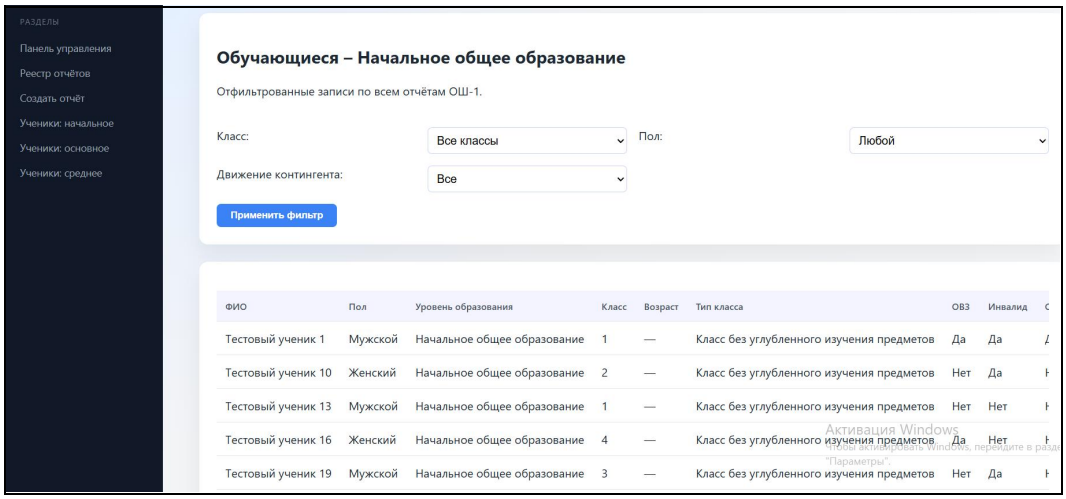


Рисунок 2.9 – Общий вид страницы создания поступления

HTML-структура страницы организована с использованием семантических тегов (Листинг 2.5).

Листинг 2.5 – HTML-структура страницы

```
      {% extends "base.html" %}

      {% block title %}Обучающиеся - {{ current_level_label }}{%
endblock %}

      {% block content %}
      <section class="card">
          <div class="flex-between">
              <div>
                  <h2>Обучающиеся - {{ current_level_label }}</h2>
                  <p>Отфильтрованные записи по всем отчётам
ОШ- 1.</p>
              </div>
          </div>
          <form method="get" class="filter-form mt-lg">
              <div class="form-grid">
      {{ filter_form.grade.label_tag }}{{ filter_form.grade }}
{{ filter_form.gender.label_tag }}{{ filter_form.gender }}
      {{ filter_form.movement_status.label_tag }}{{ filter_form.move
ment_status }}
              </div>
              <button type="submit" class="btn">Применить
фильтр</button>
          </form>
      </section>

      <section class="card mt-lg">
          <div class="table-scroll">
              <table>
                  <thead>
              <tr>
                  <th>ФИО</th>
             <th>Уровень образования</th>
                  <th>Класс</th>
                  <th>Возраст</th>
                  <th>Тип класса</th>
                  <th>ОВЗ</th>
                  <th>Инвалид</th>
                  <th>Сирота</th>
                  <th>Без попечения родителей</th>
                  <th>Движение</th>
                  <th>Форма обучения</th>
                  <th>Экстерн</th>
                  <th>Организация</th>
                  <th>Период</th>
              </tr>
              </thead>
              <tbody>
              {% for s in students %}
                  <tr>
```

```
                        <td>{{ s.full_name }}</td>
                        <td>{{ s.get_gender_display }}</td>
                        <td>{{ s.get_education_level_display }}</td>
                        <td>{{ s.grade }}</td>
                        <td>{{ s.age|default:"—" }}</td>
                        <td>{{ s.get_class_type_display }}</td>
                        <td>{{ s.has_ovz|yesno:"Да,Нет" }}</td>
                        <td>{{ s.is_disabled|yesno:"Да,Нет" }}</td>
                        <td>{{ s.is_orphan|yesno:"Да,Нет" }}</td>

<td>{{ s.without_parental_care|yesno:"Да,Нет" }}</td>
                        <td>{{ s.get_movement_status_display }}</td>
                        <td>{{ s.get_form_of_education_display }}</td>
                        <td>{{ s.is_extern|yesno:"Да,Нет" }}</td>
                        <td>{{ s.report.institution }}</td>
                        <td>{{ s.report.period }}</td>
                    </tr>
            {% empty %}
                    <tr>
                        <td colspan="15">Нет данных по выбранным
условиям.</td>
                    </tr>
                {% endfor %}
                </tbody>
            </table>

            </div>
            {% if is_paginated %}
                <div class="pagination">
                    {% if page_obj.has_previous %}
                        <a
href="?page={{ page_obj.previous_page_number }}">&laquo;</a>
                    {% endif %}

        <span>Стр. {{ page_obj.number }} из
{{ page_obj.paginator.num_pages }}</span>
                    {% if page_obj.has_next %}
                        <a
href="?page={{ page_obj.next_page_number }}">&raquo;</a>
                    {% endif %}
                </div>
            {% endif %}
        </section>
        {% endblock %}
```

Функциональность интерфейса реализована на языке Python. При загрузке страницы выполняются асинхронные запросы к API для получения данных для таблицы. В листинге 2.6 приведен код инициализации формы.

Листинг 2.6 – Функции загрузки таблицы

```
        @login_required
```

```
def report_students(request, pk):
    report = get_object_or_404(
        OshReport.objects.select_related("institution",
"period"), pk=pk
    )
    students = (
        Student.objects.filter(report=report)
        .order_by("education_level", "grade", "full_name")
        .all()
    )

    if request.method == "POST":
        form = StudentForm(request.POST)
        if form.is_valid():
            student = form.save(commit=False)
            student.report = report
            student.save()
            messages.success(request, "Ученик добавлен.")
        return redirect("report_students", pk=report.pk)
    else:
        form = StudentForm()

    return render(
        request,
        "osh1/report_students.html",
        {
            "report": report,
            "students": students,
            "form": form,
        },
    )
```

Интерфейс обеспечивает удобное взаимодействие с пользователем через интуитивно понятные элементы управления, мгновенную обратную связь и защиту от некорректного ввода данных. Все операции выполняются без перезагрузки страницы, что повышает удобство работы с системой.

Инструкция по работе с программой

Авторизация в систему.

1) Вход в систему. Для начала работы с системой необходимо выполнить вход под своей учетной записью. На странице авторизации требуется ввести логин и пароль, после чего нажать кнопку «Войти» (Рисунок 2.10).

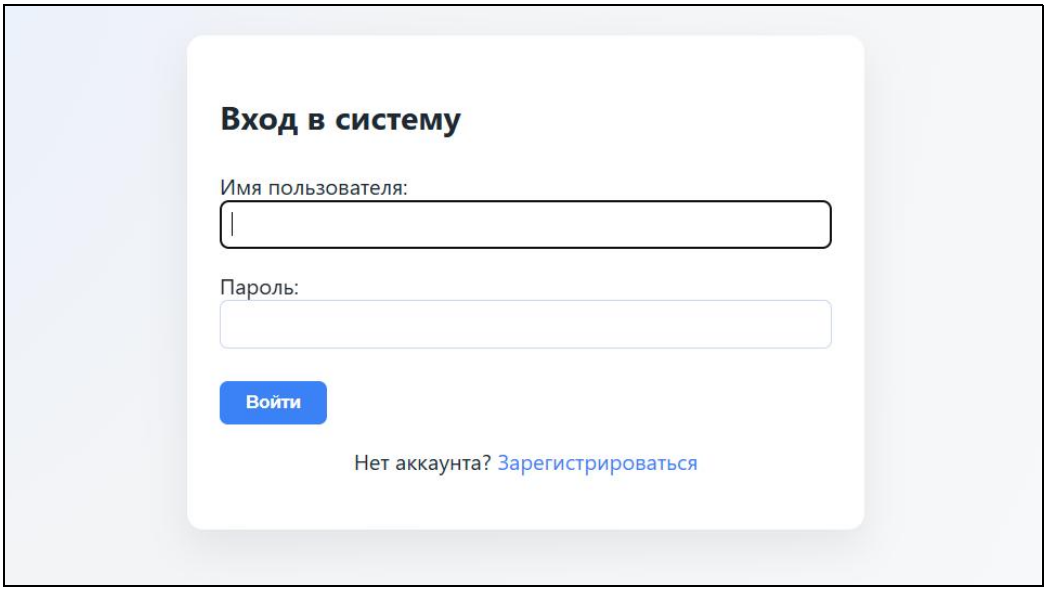


Рисунок 2.10 – Страница «Авторизация»

При ошибке в данных появляется соответствующее уведомление. Успешная аутентификация автоматически перенаправляет пользователя на главную страницу системы.

2) Регистрация нового пользователя. Если учетная запись отсутствует, необходимо пройти регистрацию, нажав на странице входа ссылку «Зарегистрироваться». В форме регистрации нужно заполнить обязательные поля: ФИО, email, логин (только латинские символы), пароль (не менее 6 символов) и подтверждение пароля. Дополнительно можно выбрать подразделение из выпадающего списка. Важно убедиться, что пароли в обоих полях совпадают, а email указан в корректном формате (Рисунок 2.11).

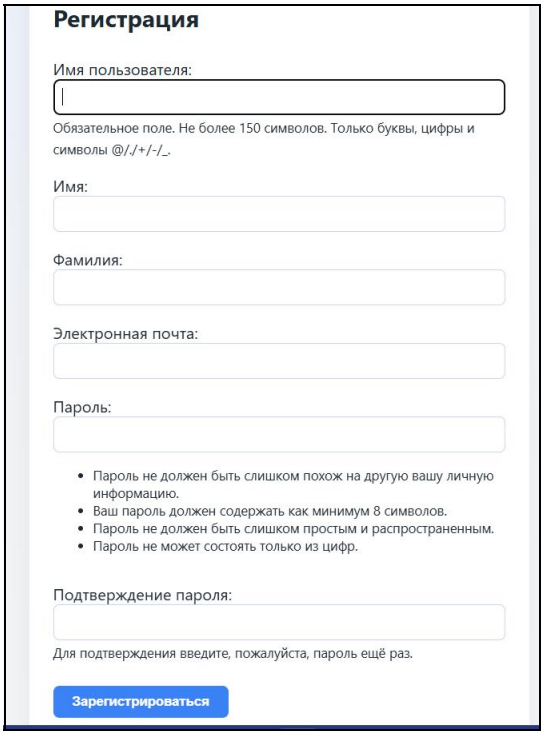


Рисунок 2.11 – Страница «Регистрации»

3) Безопасность и доступ. Рекомендуется использовать сложные пароли, содержащие буквы, цифры и специальные символы. Учетные данные не следует передавать третьим лицам. По окончании работы необходимо закрывать браузер для завершения сеанса.

Главная страница и навигация.

После успешного входа в систему открывается главная страница. Интерфейс разделен на две основные части: слева расположена панель навигации, справа - рабочая область с информацией (Рисунок 2.12).

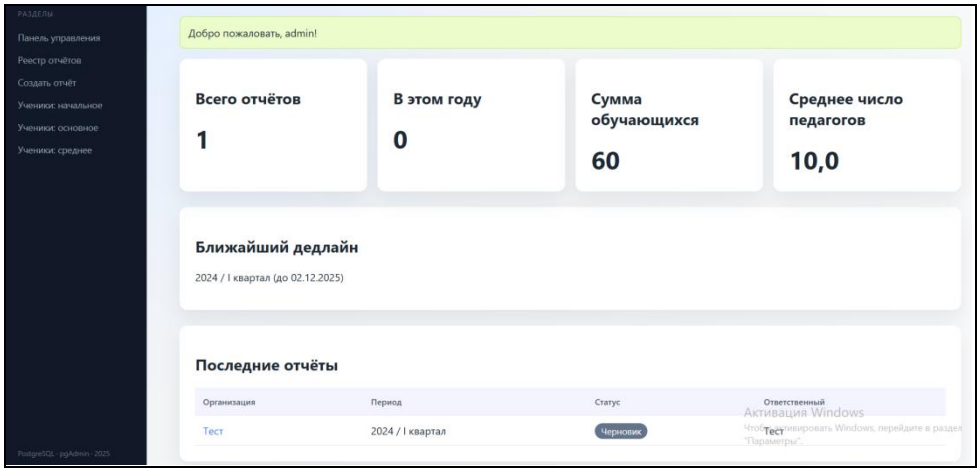


Рисунок 2.12 – Главная страница

Панель навигации содержит все основные разделы системы:

1) Панель управления– возврат на стартовую страницу;
2) Реестр отчетов - здесь содержатся все отчеты;
3) Создать отчет– здесь можно создать отчет;
4) Разделы с информацией об учениках.

Рабочая область главной страницы отображает ключевые показатели в виде информационных карточек:

1) всего отчетов;
2) отчетов в этом году;
3) всего обучающихся;
4) среднее число педагогов;
5) ближайший дедлайн;
6) последние отчеты;

В верхнем правом углу отображается информация о текущем пользователе. Нажатие на имя пользователя появляется кнопка для выхода из системы, показана на рисунке 2.13.

Рисунок 2.13 – Кнопка выхода из системы

Создание отчета. Раздел «создание отчета» предназначен для составления отчета (Рисунок 2.14).

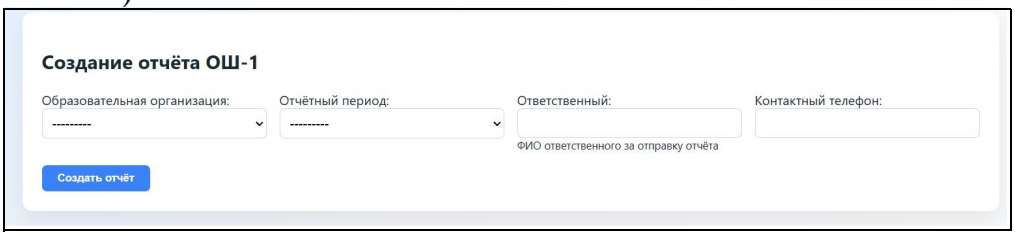
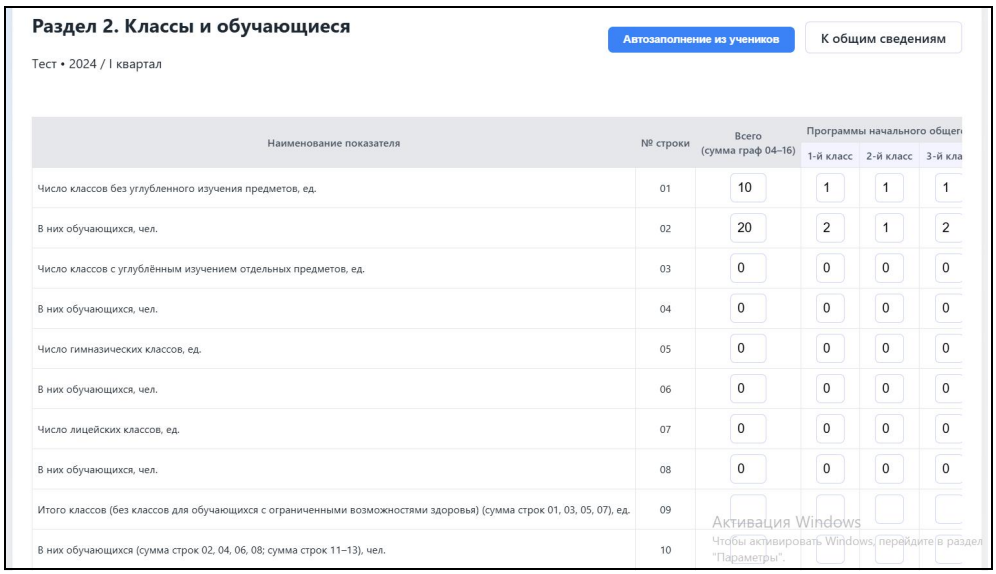


Рисунок 2.14 – Раздел «Создание отчета»

Для добавления нового отчета необходимо заполнить поля и нажать кнопку «Создать отчет», после чего появится таблица , для заполнения отчета. (Рисунок 2.15).



Раздел 2. Классы и обучающиеся

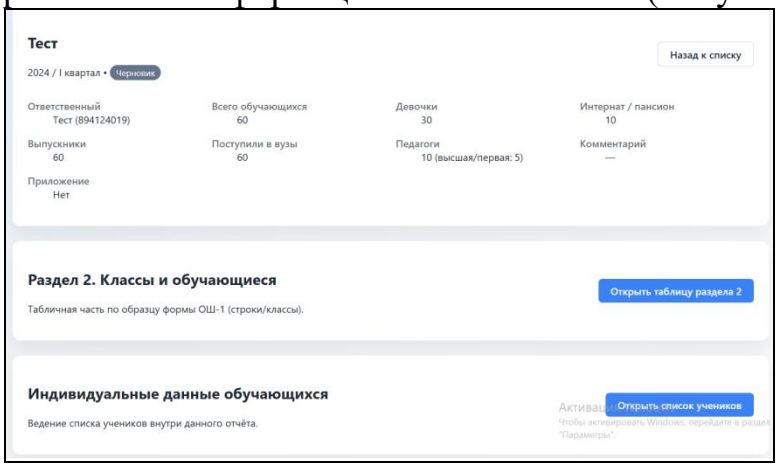
Тест • 2024 / I квартал

Автозаполнение из учеников | К общим сведениям

Наименование показателя	№ строки	Всего (сумма граф 04–16)	Программы начального общег		
			1-й класс	2-й класс	3-й кла
Число классов без углубленного изучения предметов, ед.	01	10	1	1	1
В них обучающихся, чел.	02	20	2	1	2
Число классов с углублённым изучением отдельных предметов, ед.	03	0	0	0	0
В них обучающихся, чел.	04	0	0	0	0
Число гимназических классов, ед.	05	0	0	0	0
В них обучающихся, чел.	06	0	0	0	0
Число лицейских классов, ед.	07	0	0	0	0
В них обучающихся, чел.	08	0	0	0	0
Итого классов (без классов для обучающихся с ограниченными возможностями здоровья) (сумма строк 01, 03, 05, 07), ед.	09				
В них обучающихся (сумма строк 02, 04, 06, 08; сумма строк 11–13), чел.	10				

Рисунок 2.15 – Таблица для отчета

Отчет можно редактировать в ручную, а можно нажать кнопку «Автозаполнение учеников», что автоматически заполнит поля. Кнопка «К общим сведениям», где содержится вся информация об этом отчете. (Рисунок 2.16).



Тест

2024 / I квартал • Черновик

Назад к списку

Ответственный: Тест (894124019)
Всего обучающихся: 60
Девочки: 30
Интернат / пансион: 10
Выпускники: 60
Поступили в вузы: 60
Педагоги: 10 (высшая/первая: 5)
Комментарий: —
Приложение: Нет

Раздел 2. Классы и обучающиеся

Табличная часть по образцу формы ОШ-1 (строки/классы).

Открыть таблицу раздела 2

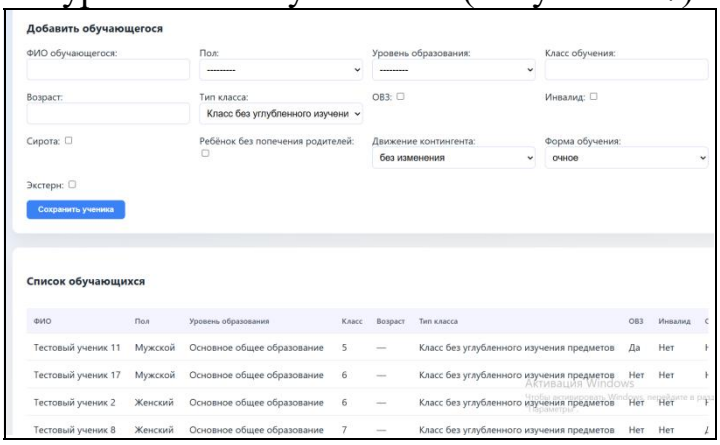
Индивидуальные данные обучающихся

Ведение списка учеников внутри данного отчёта.

Открыть список учеников

Рисунок 2.16 – Общие сведения

Мы также можем посмотреть данные об учениках, учавствовавших в отчете нажав кнопку «Откурыть списки учеников» (Рисунок 2.17).



Добавить обучающегося

ФИО обучающегося: | Пол: --------- | Уровень образования: --------- | Класс обучения:

Возраст: | Тип класса: Класс без углубленного изучени | ОВЗ: ☐ | Инвалид: ☐

Сирота: ☐ | Ребёнок без попечения родителей: ☐ | Движение контингента: без изменения | Форма обучения: очное

Экстерн: ☐

Сохранить ученика

Список обучающихся

ФИО	Пол	Уровень образования	Класс	Возраст	Тип класса	ОВЗ	Инвалид
Тестовый ученик 11	Мужской	Основное общее образование	5	—	Класс без углубленного изучения предметов	Да	Нет
Тестовый ученик 17	Мужской	Основное общее образование	6	—	Класс без углубленного изучения предметов	Нет	Нет
Тестовый ученик 2	Женский	Основное общее образование	6	—	Класс без углубленного изучения предметов	Нет	Нет
Тестовый ученик 8	Женский	Основное общее образование	7	—	Класс без углубленного изучения предметов	Нет	Нет

Рисунок 2.17 – Списки учеников

Здесь можно просматривать данные об учениках, редактировать информацию и добавлять новых учеников.

Новый раздел «реестр отчетов» позволяет посмотреть все существующие отчеты, также сюда добавлена фильтрация, для более удобной навигации (Рисунок 2.18).

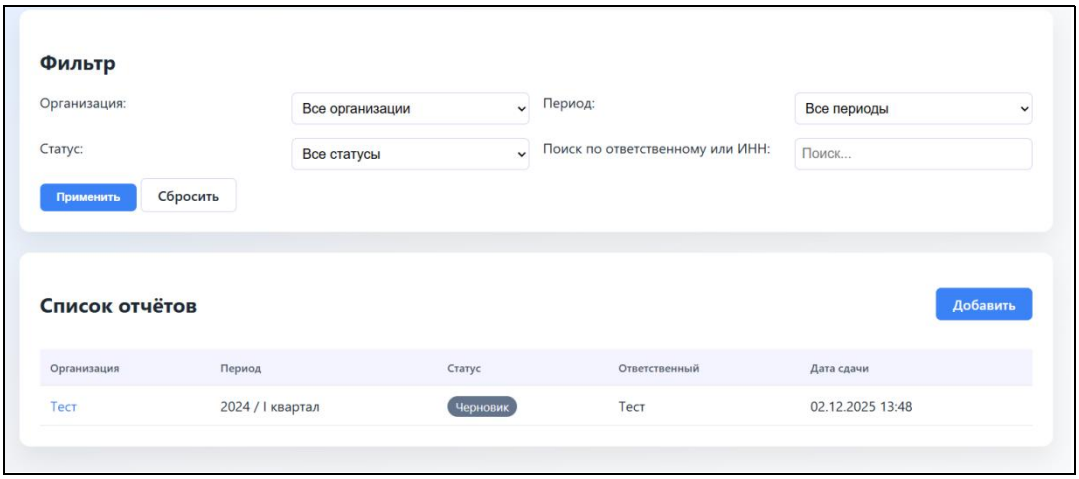


Рисунок 2.18 – Раздел «Реестр отчетов»

2.9 Доработка приложения

2.9.1 Разработка блок-схем функций

Первым делом следует добавить модуль экспорта таблицы в Excel-формат. Наличие инструмента экспорта превращает систему из простого "редактора формы" в полноценную аналитическую платформу. Это напрямую повышает удовлетворенность пользователей и снижает необходимость в обходных путях и дублировании данных.

Также тследует добавить модуль генерации печатной формы, так как несмотря на переход на электронный документооборот, школа обязана по внутренним регламентам или по запросу проверяющих органов хранить бумажные копии отчетов, подписанные собственноручно. Сформированный PDF предназначен для распечатки и подшивки в архив.

Модуль аудита и логирования действий требуется добавить согласно ФЗ-152, т.к. любые организации обязаны обеспечивать контроль за обработкой персональных и статистических данных. Система аудита является техническим средством, обеспечивающим выполнение этого требования, и ее наличие может проверяться. Сами же логи безопасности позволяют выявлять подозрительную активность (многочисленные неудачные попытки входа, доступ с необычных IP-адресов), предотвращая потенциальные инциденты информационной безопасности.

Для начала необходимо создать блок-схему функции, чтобы лучше понимать принцип ее будущей работы (Рисунок 2.19).

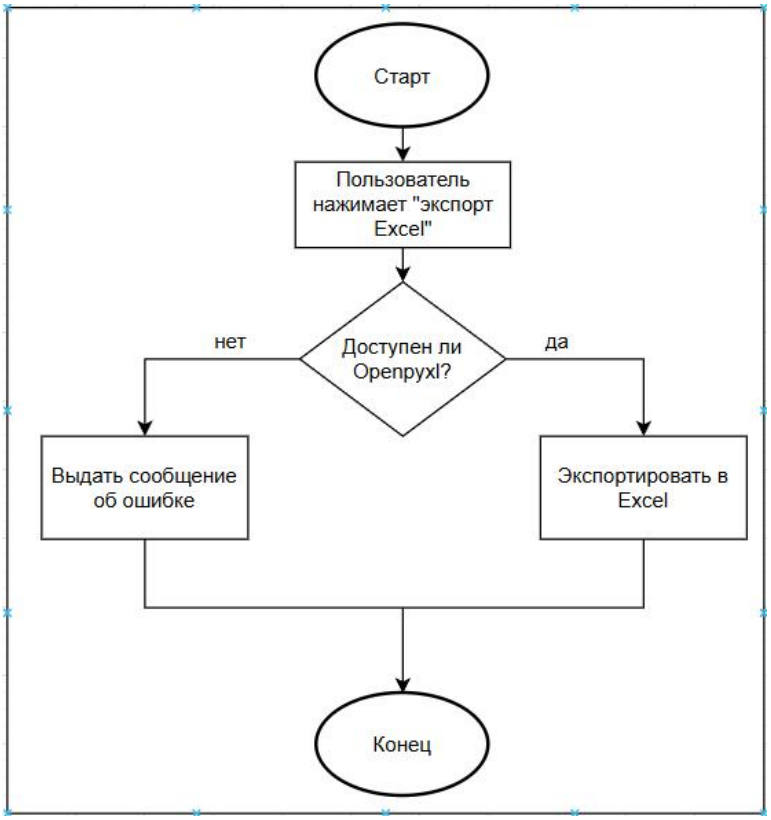


Рисунок 2.19 – Блок-схема экспорта Excel

Данная блок-схема описывает последовательность действий при запуске пользователем процедуры экспорта данных в формат Excel. Процесс начинается с инициации операции, когда пользователь нажимает кнопку «Экспорт Excel» в интерфейсе приложения. После этого система автоматически выполняет проверку наличия необходимой программной библиотеки openpyxl, которая требуется для генерации файлов в формате .xlsx. Если проверка показывает, что библиотека недоступна (например, из-за ошибки установки или конфигурации сервера), процесс прерывается, и пользователю отображается сообщение об ошибке, после чего работа алгоритма завершается. Если же библиотека успешно обнаружена, система переходит непосредственно к этапу экспорта: формирует структурированный файл Excel на основе текущих данных отчёта и предоставляет его пользователю для загрузки, после чего процесс также завершается.

Следующая блок-схема будет описывать процесс работы функции генерации печатной формы (Рисунок 2.20).

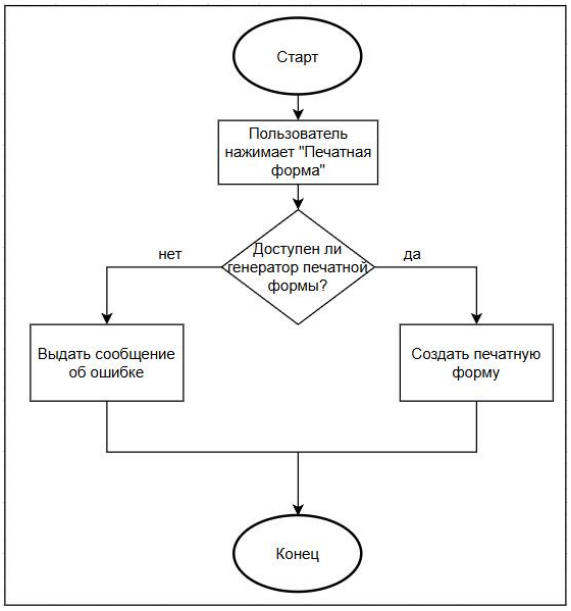


Рисунок 2.20 – Блок-схема генерации печатной формы

Данная блок-схема иллюстрирует процесс создания печатной формы документ. Начало процесса происходит, когда пользователь нажимает кнопку «Печатная форма» в интерфейсе приложения. Далее система автоматически проверяет доступность и работоспособность специального модуля — генератора печатной формы, который отвечает за преобразование данных в готовый для печати или сохранения документ (часто в формате PDF). Если генератор по какой-либо причине недоступен (например, отсутствует необходимое программное обеспечение на сервере, закончилась лицензия или возникла внутренняя ошибка), процесс прерывается, и пользователю показывается сообщение об ошибке с пояснением о невозможности выполнить операцию. В случае успешной проверки система переходит к этапу генерации: данные текущего отчёта форматируются согласно официальному макету, добавляются необходимые реквизиты, и итоговый файл формируется для последующей загрузки пользователем. После этого процесс завершается.

Следующая блок-схема описывает процесс аудита и логирования действий (Рисунок 2.21).

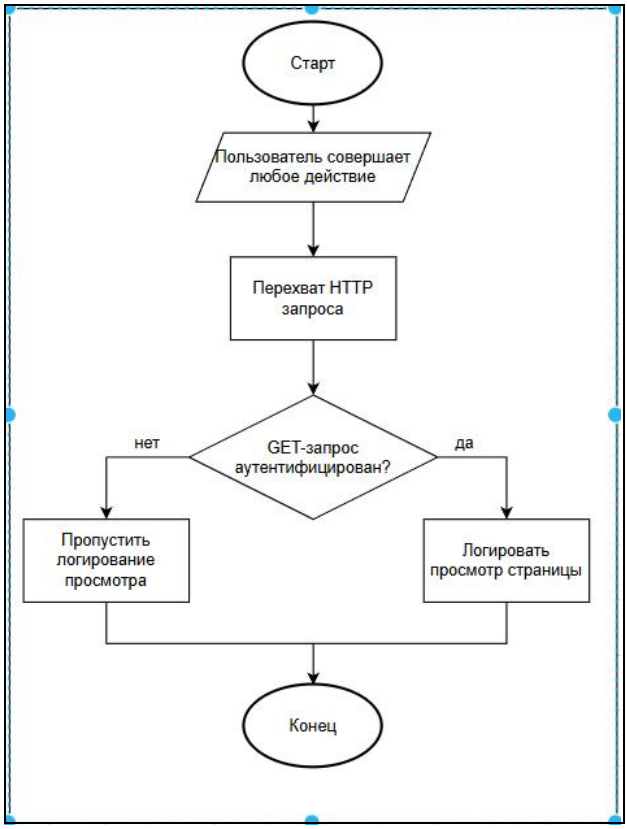
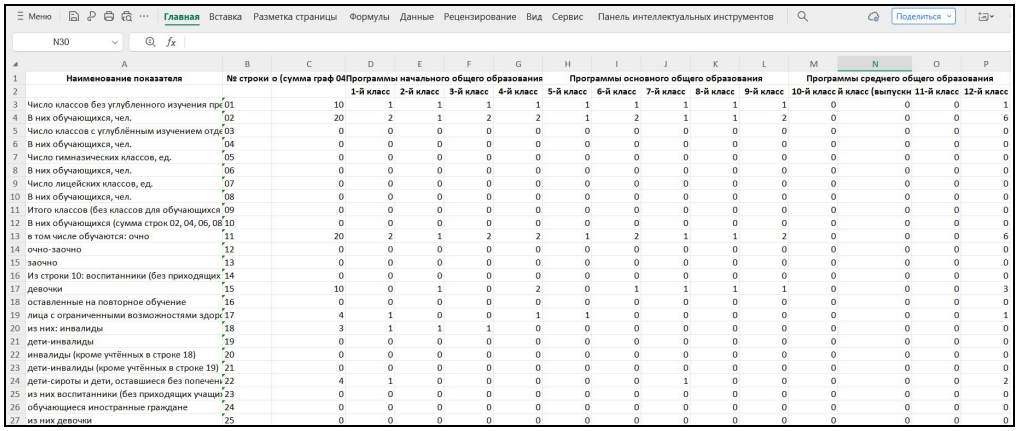


Рисунок 2.21 – Блок-схема аудита и логирования действий

Процесс запускается при любом действии пользователя в приложении, которое формирует HTTP-запрос к серверу. Первым шагом система перехватывает этот запрос для анализа. Далее выполняется проверка двух условий: является ли метод запроса GET (то есть запрос на получение данных, а не их изменение) и прошёл ли пользователь успешную аутентификацию (вошёл в систему под своей учётной записью). Если оба условия выполнены — запрос является GET-запросом от аутентифицированного пользователя — система фиксирует факт просмотра страницы в журнале аудита. Это позволяет отслеживать, какие разделы интерфейса и данные просматривал пользователь.

2.9.2 Разработка функционала

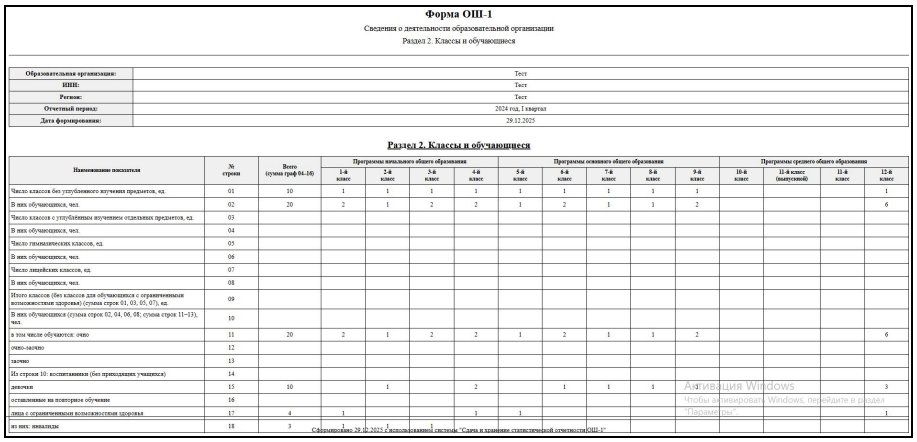
Для начала необходимо реализовать функцию для экспорта в Excel. Для этого будут использоваться инструменты PyCharm с библиотекой Openpyxl и язык программирования python. Данный модуль предоставляет функционал гибкой выгрузки любых данных из системы в формате Microsoft Excel (.xlsx). (Рисунок 2.22)



Наименование показателя	№ строки	о (сумма граф 04	Программы начального общего образования				Программы основного общего образования					Программы среднего общего образования			
			1-й класс	2-й класс	3-й класс	4-й класс	5-й класс	6-й класс	7-й класс	8-й класс	9-й класс	10-й класс	й класс (выпускн	11-й класс	12-й класс
Число классов без углубленного изучения пре	01	10	1	1	1	1	1	1	1	1	1	0	0	0	1
В них обучающихся, чел.	02	20	2	1	2	2	1	2	1	1	2	0	0	0	6
Число классов с углублённым изучением отде	03	0	0	0	0	0	0	0	0	0	0	0	0	0	0
В них обучающихся, чел.	04	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Число гимназических классов, ед.	05	0	0	0	0	0	0	0	0	0	0	0	0	0	0
В них обучающихся, чел.	06	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Число лицейских классов, ед.	07	0	0	0	0	0	0	0	0	0	0	0	0	0	0
В них обучающихся, чел.	08	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Итого классов (без классов для обучающихся	09	0	0	0	0	0	0	0	0	0	0	0	0	0	0
В них обучающихся (сумма строк 02, 04, 06, 08	10	0	0	0	0	0	0	0	0	0	0	0	0	0	0
в том числе обучаются: очно	11	20	2	1	2	2	1	2	1	1	2	0	0	0	6
очно-заочно	12	0	0	0	0	0	0	0	0	0	0	0	0	0	0
заочно	13	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Из строки 10: воспитанники (без приходящих	14	0	0	0	0	0	0	0	0	0	0	0	0	0	0
девочки	15	10	0	1	0	2	0	1	1	1	1	0	0	0	3
оставленные на повторное обучение	16	0	0	0	0	0	0	0	0	0	0	0	0	0	0
лица с ограниченными возможностями здоро	17	4	1	0	0	1	1	0	0	0	0	0	0	0	1
из них: инвалиды	18	3	1	1	1	0	0	0	0	0	0	0	0	0	0
дети-инвалиды	19	0	0	0	0	0	0	0	0	0	0	0	0	0	0
инвалиды (кроме учтённых в строке 18)	20	0	0	0	0	0	0	0	0	0	0	0	0	0	0
дети-инвалиды (кроме учтённых в строке 19)	21	0	0	0	0	0	0	0	0	0	0	0	0	0	0
дети-сироты и дети, оставшиеся без попечен	22	4	1	0	0	0	0	0	1	0	0	0	0	0	2
из них воспитанники (без приходящих учащи	23	0	0	0	0	0	0	0	0	0	0	0	0	0	0
обучающиеся иностранные граждане	24	0	0	0	0	0	0	0	0	0	0	0	0	0	0
из них девочки	25	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Рисунок 2.22 - Экспорт в Excel

Функция генерации печатной формы реализована в форме PDF для печати (Рисунок 2.23)



Форма ОШ-1

Сведения о деятельности образовательной организации

Раздел 2. Классы и обучающиеся

Образовательная организация:	Тест
ИНН:	Тест
Регион:	Тест
Отчетный период:	2024 год, I квартал
Дата формирования:	29.12.2025

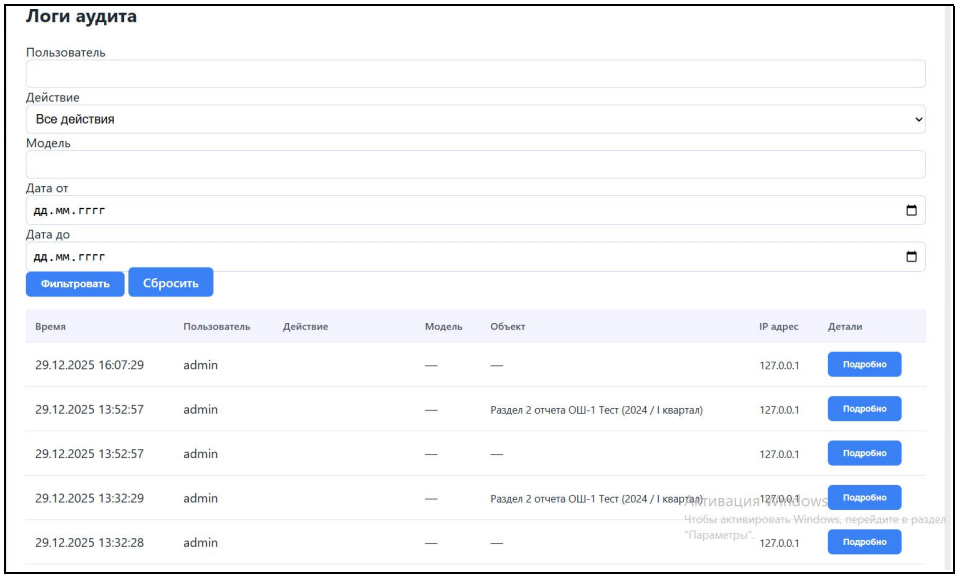
Раздел 2. Классы и обучающиеся

Наименование показателя	№ строки	Всего (сумма граф 04-10)	Программы начального общего образования				Программы основного общего образования					Программы среднего общего образования			
			1-й класс	2-й класс	3-й класс	4-й класс	5-й класс	6-й класс	7-й класс	8-й класс	9-й класс	10-й класс	11-й класс (выпускной)	11-й класс	12-й класс
Число классов без углубленного изучения предметов, ед.	01	10	1	1	1	1	1	1	1	1	1				1
В них обучающихся, чел.	02	20	2	1	2	2	1	2	1	1	2				6
Число классов с углублённым изучением отдельных предметов, ед.	03														
В них обучающихся, чел.	04														
Число гимназических классов, ед.	05														
В них обучающихся, чел.	06														
Число лицейских классов, ед.	07														
В них обучающихся, чел.	08														
Итого классов (без классов для обучающихся с ограниченными возможностями здоровья) (сумма строк 01, 03, 05, 07), ед.	09														
В них обучающихся (сумма строк 02, 04, 06, 08; сумма строк 11–13), чел.	10														
в том числе обучаются: очно	11	20	2	1	2	2	1	2	1	1	2				6
очно-заочно	12														
заочно	13														
Из строки 10: воспитанники (без приходящих учащихся)	14														
девочки	15	10		1		2		1	1	1					3
оставленные на повторное обучение	16														
лица с ограниченными возможностями здоровья	17	4	1			1	1								1
из них: инвалиды	18	3	1		1										

Сформировано 29.12.2025 Автоматизированная система "Сдача и хранение статистической отчетности ОШ-1"

Рисунок 2.23- Генерация печатной формы

Модуль, показанный на рисунке 2.24 создан для отслеживания администратором действий, происходящих на сайте, в том числе и несанкционированных



Логи аудита

Пользователь

Действие: Все действия

Модель

Дата от: дд.мм.гггг

Дата до: дд.мм.гггг

Фильтровать | Сбросить

Время	Пользователь	Действие	Модель	Объект	IP адрес	Детали
29.12.2025 16:07:29	admin		—	—	127.0.0.1	Подробно
29.12.2025 13:52:57	admin		—	Раздел 2 отчета ОШ-1 Тест (2024 / I квартал)	127.0.0.1	Подробно
29.12.2025 13:52:57	admin		—	—	127.0.0.1	Подробно
29.12.2025 13:32:29	admin		—	Раздел 2 отчета ОШ-1 Тест (2024 / I квартал)	127.0.0.1	Подробно
29.12.2025 13:32:28	admin		—	—	127.0.0.1	Подробно

Рисунок 2.24- Логи аудита

Заключение

В процессе прохождения учебной практики все поставленные цели и задачи были успешно достигнуты. Основным результатом проделанной работы стала полностью готовая программа и пакет технической документации для сдачи демонстрационного экзамена, что также послужило надежной практической базой для реализации модуля дипломного проекта.

Для достижения этого результата был пройден полный цикл разработки информационной системы:

- спроектирована структура базы данных и написан функциональный программный код;
- настроена система безопасности с разграничением прав доступа пользователей по ролям;
- разработан удобный, интуитивно понятный и защищенный пользовательский интерфейс;
- составлена и оформлена вся необходимая проектная документация.

Все этапы создания проекта последовательно фиксировались в системе контроля версий Git. Итоговый программный продукт функционирует стабильно, без сбоев и эффективно решает реальные бизнес-задачи организации.

Библиография

1 Django Software Foundation. Django 4.x Documentation — подробная справка по моделям, формам, авторизации, шаблонам. https://docs.djangoproject.com/

2 PostgreSQL Global Development Group. PostgreSQL Documentation — работа с БД, типы данных, оптимизация. https://www.postgresql.org/docs/

3 V. V. Рубцов. Django 4.0 Практика создания веб-приложений — книга по практической разработке на Django (изд. Питер).

4 MDN Web Docs: Веб-технологии для разработчиков (HTML, CSS, JavaScript) [Электронный ресурс] – Режим доступа: https://developer.mozilla.org/ru/;

5 Руководство пользователя инструмента администрирования баз данных pgAdmin 4 [Электронный ресурс] – Режим доступа: https://www.pgadmin.org/docs/;

6 Официальная документация системы контроля версий Git [Электронный ресурс] – Режим доступа: https://git-scm.com/doc;

7 ГОСТ 19.701-90 ЕСПД. Схемы алгоритмов, программ, данных и систем. Условные обозначения и правила выполнения [Электронный ресурс] – Режим доступа: https://docs.cntd.ru/document/1200007206.